

Language Models: After the Transformer

Dongwook Choi

Language & AGI Lab, Yonsei University

2025. 06. 17

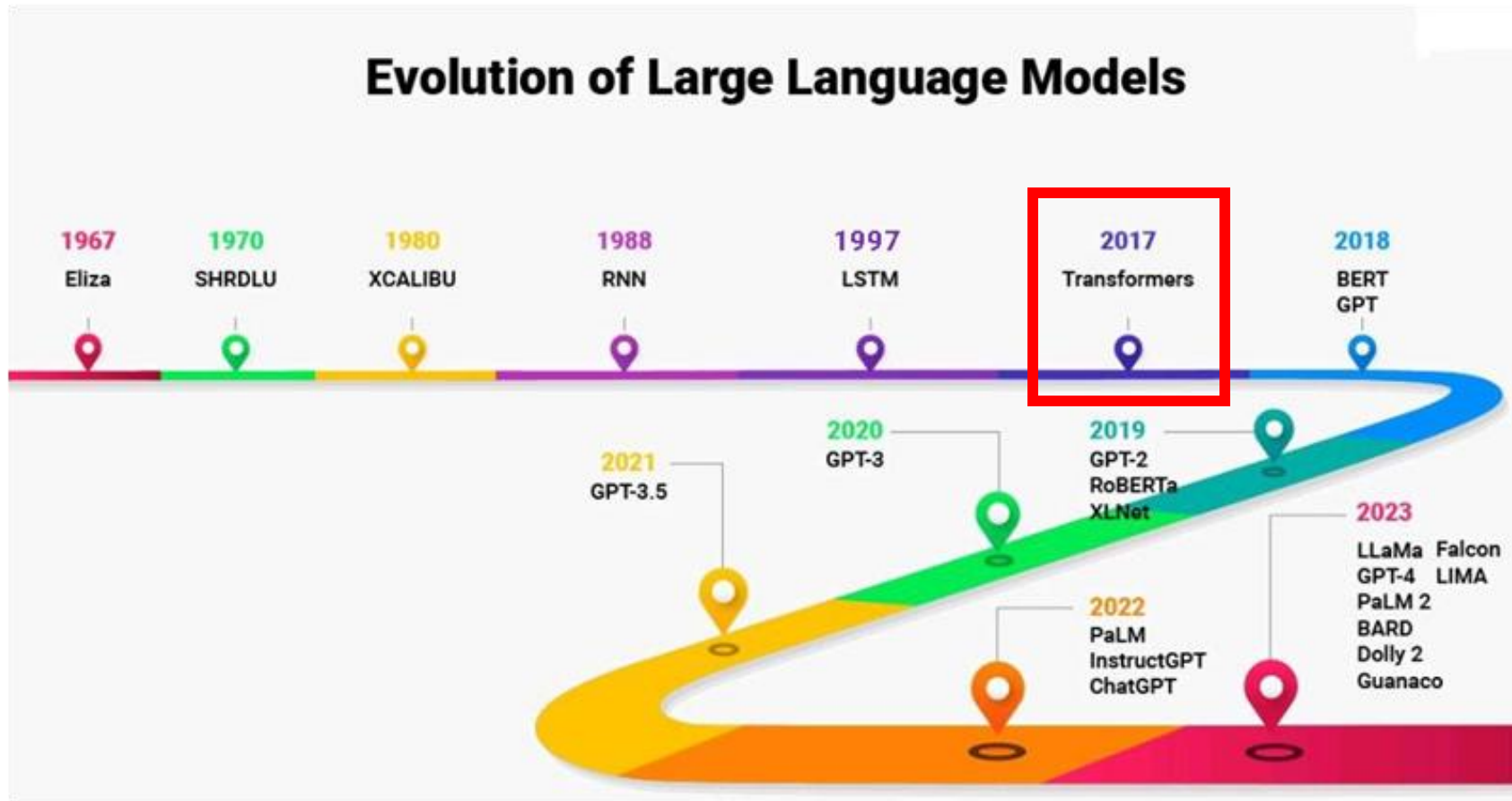
Contents

1. Transformer
2. Natural Language Understanding & BERT
3. Natural Language Generation & GPT
4. 최근의 Language Models

실습 코드

https://drive.google.com/file/d/1ktzWde-tsCOr22gW9A53YDX7KWMs_2tp/view?usp=sharing

Transformer



Transformer 이후, 언어 모델의 성능과 기술은 빠르게 발전.

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

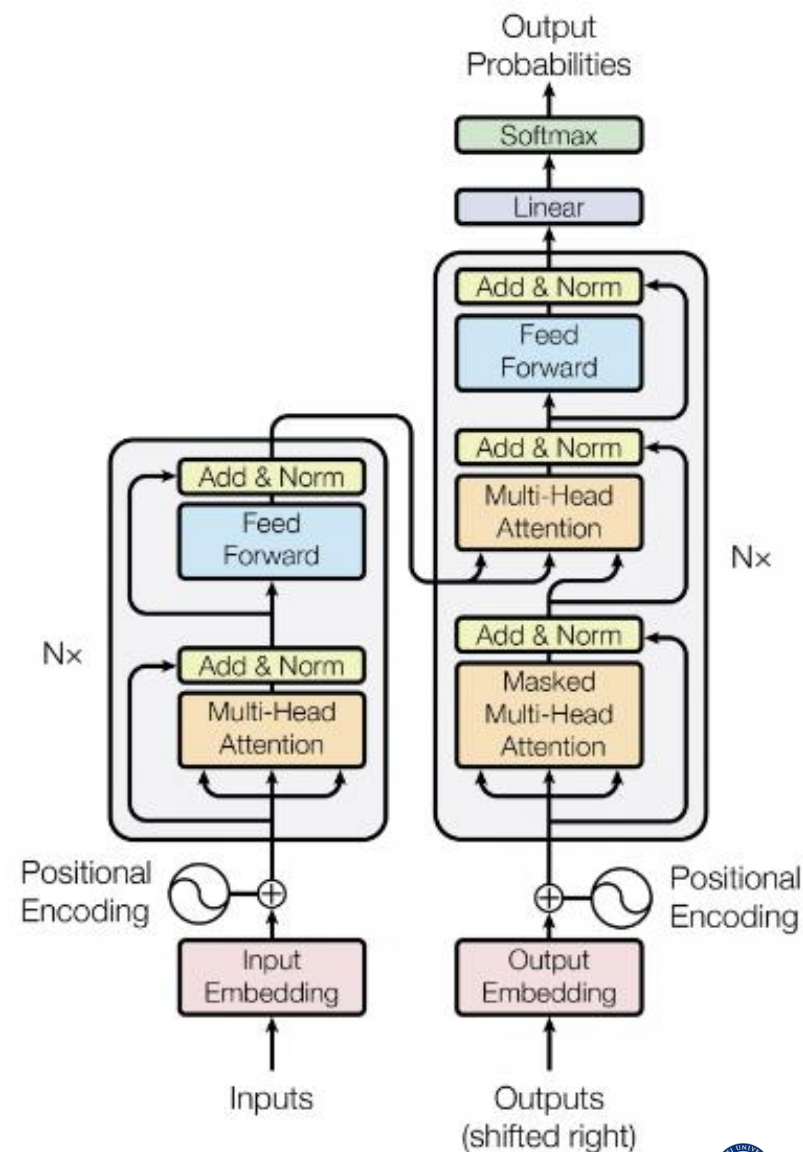
Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

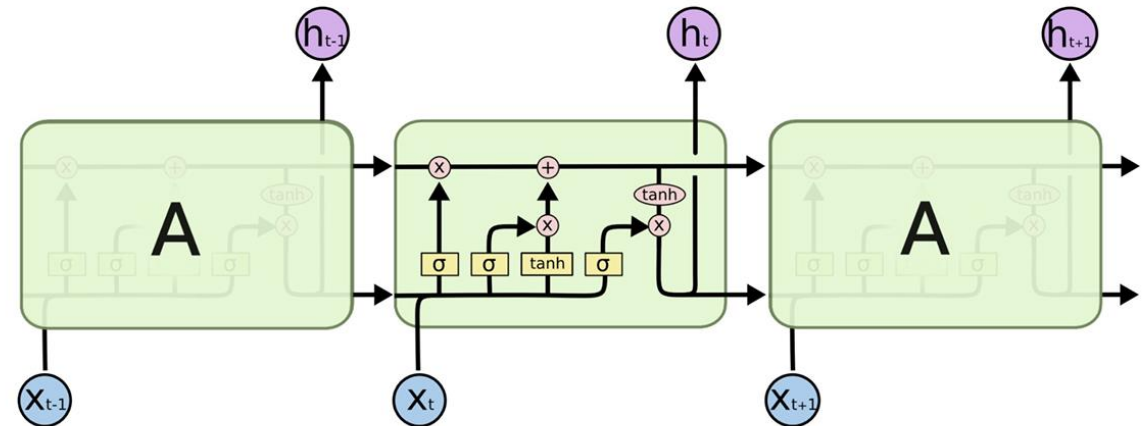
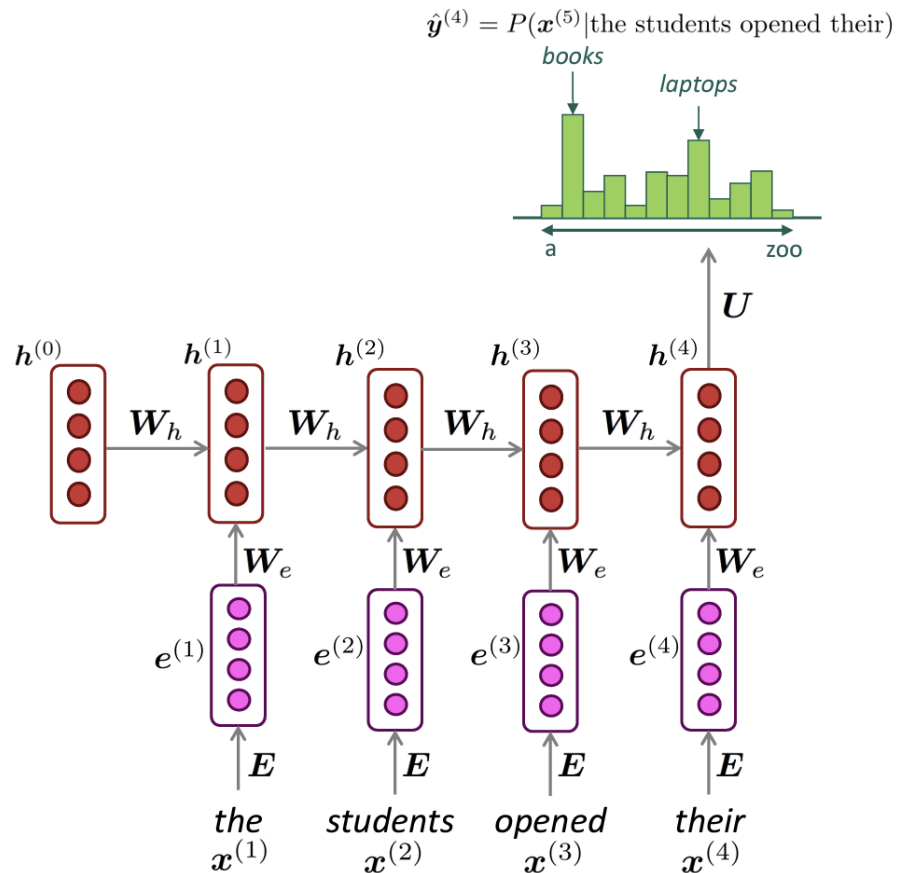
Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

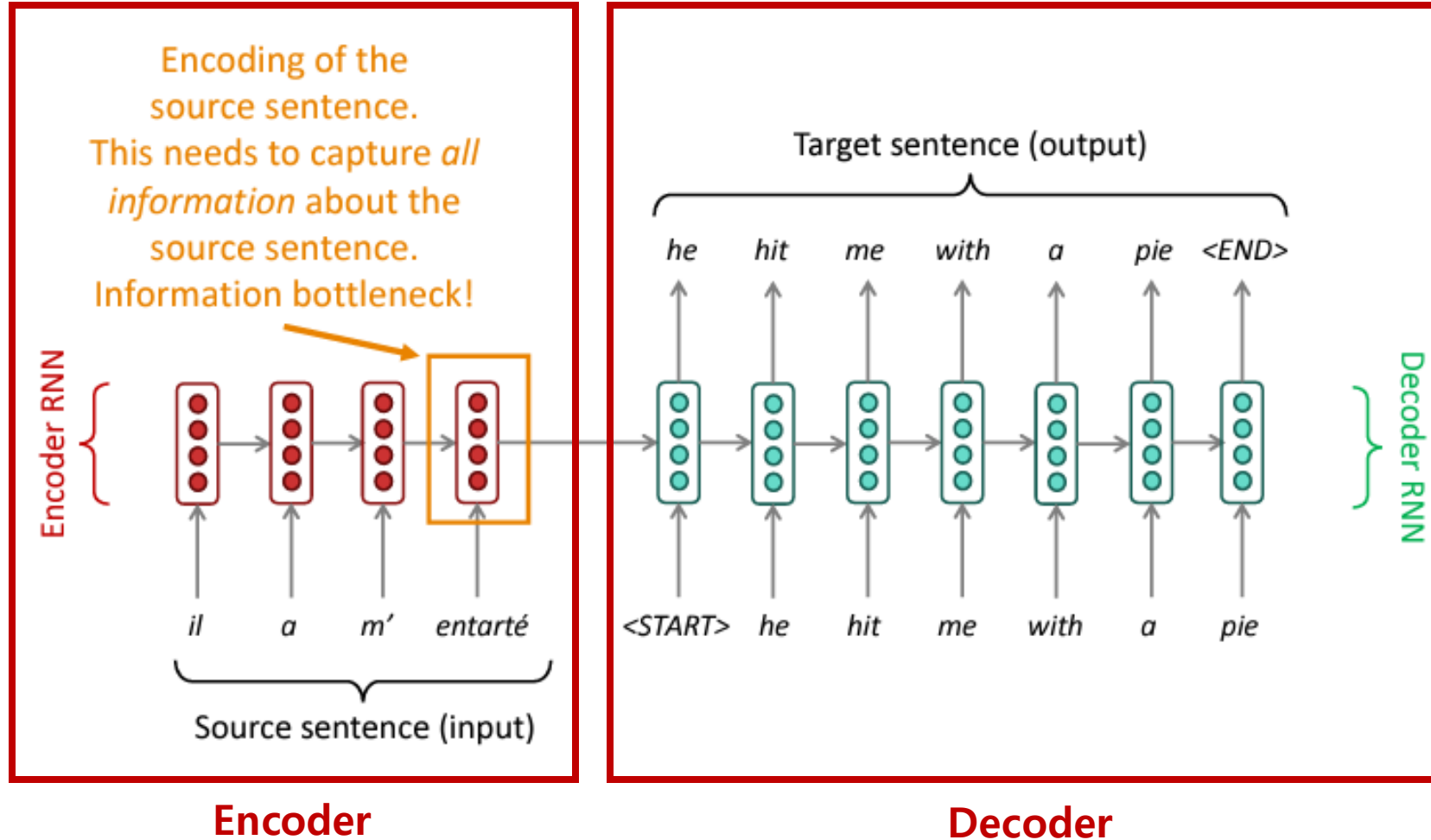


Recap: RNN, LSTM

- 단일 cell로 구성된 언어 모델
- 순차적인 연산을 통해 sequential data 처리

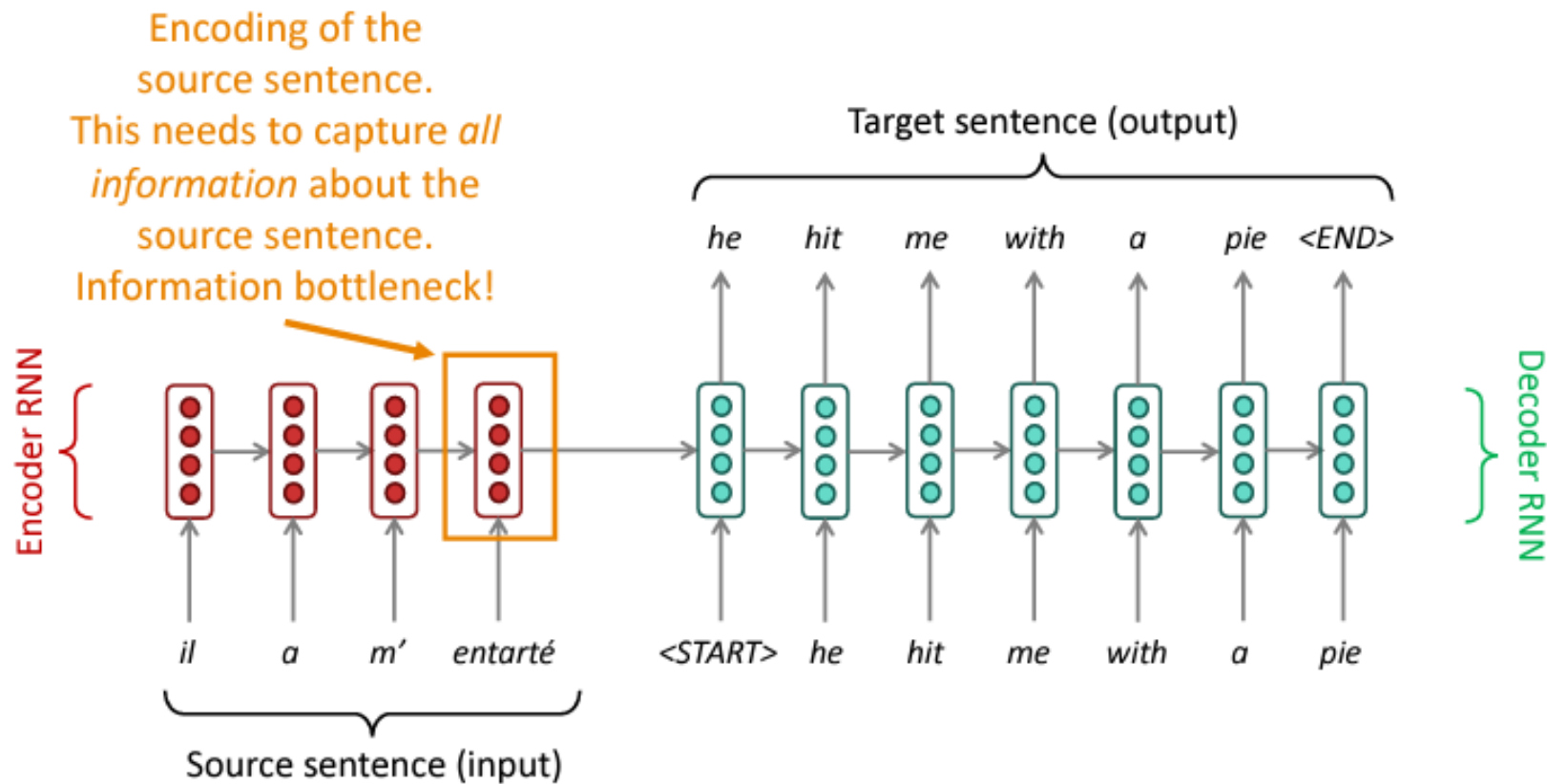


Recap: Seq2Seq



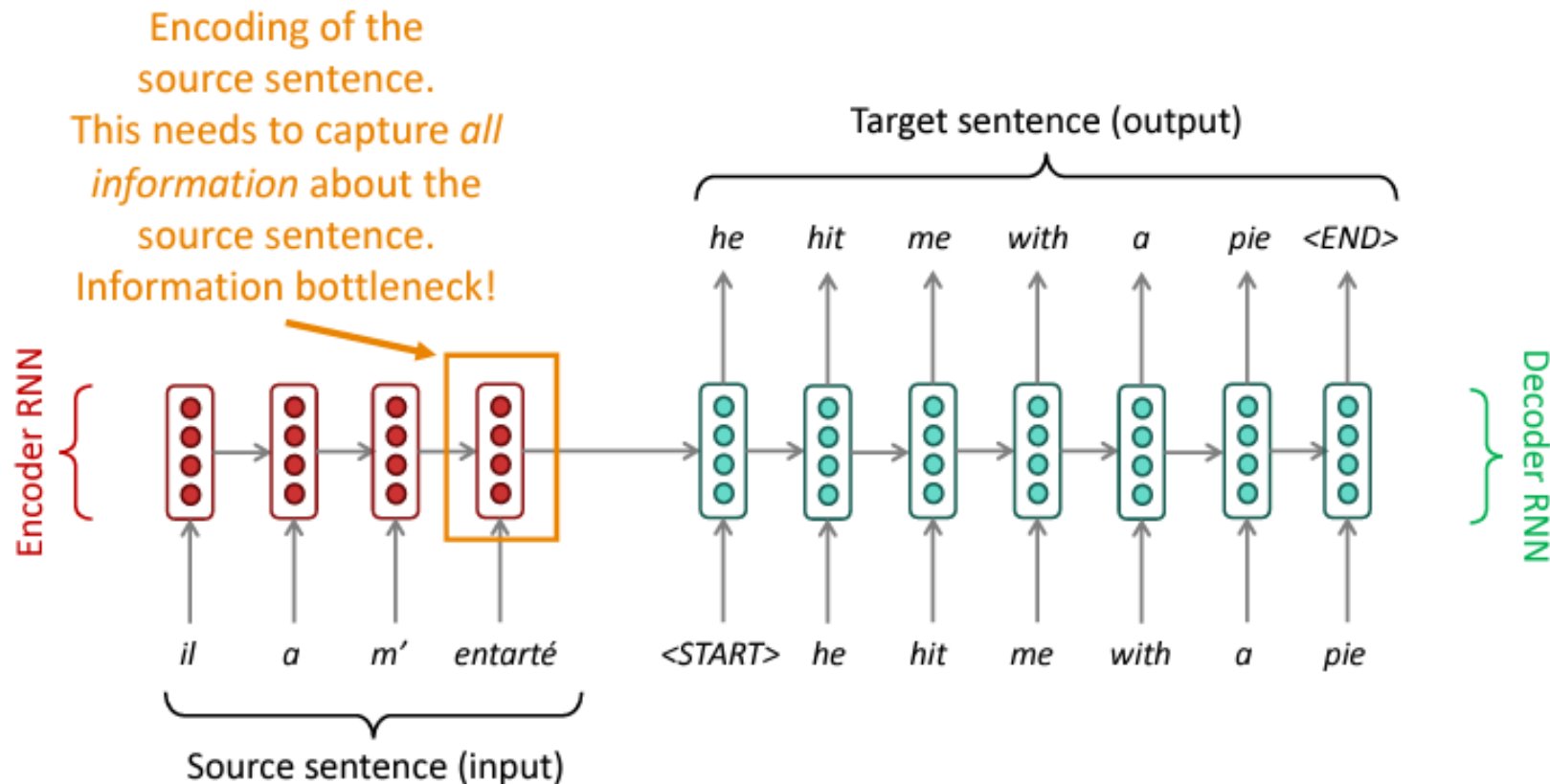
Recap: Seq2Seq

- Encoder – Decoder 구조의 언어 모델
- 기존 RNN과는 달리 input sequence 전체의 정보(맥락)를 통해 output sequence를 생성할 수 있음



Recap: the bottleneck problem

- 마지막 hidden state vector(=context vector)는 input sequence의 모든 정보를 포함하고 있음
- 하지만, hidden state vector의 크기는 고정이므로, input sequence가 길어질수록 병목 현상



Recap: Attention

- 번역(출력) 시에 문장의 특정 부분에 집중(Attention)하는 방법 사용

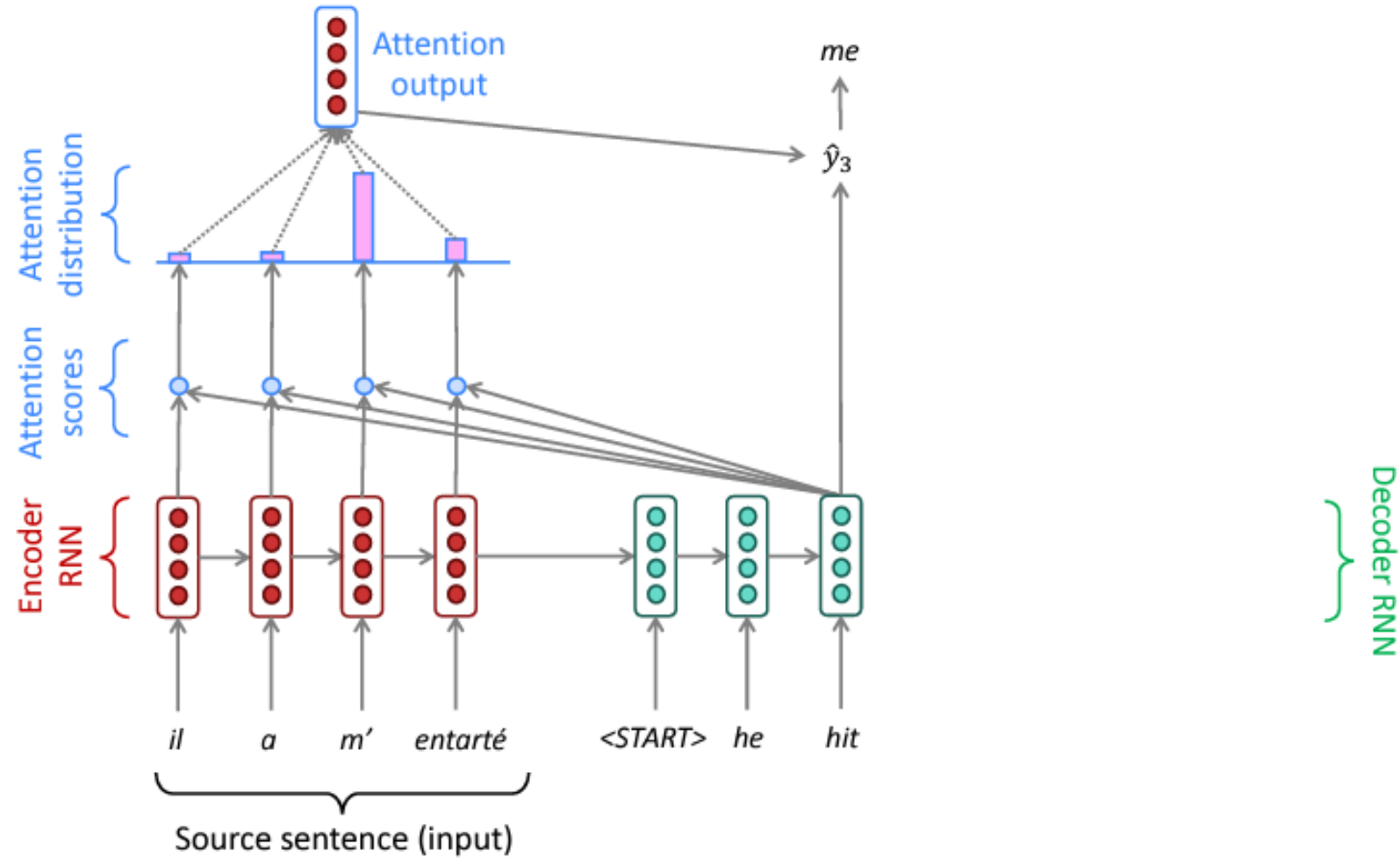
번역(Machine translation) 예시

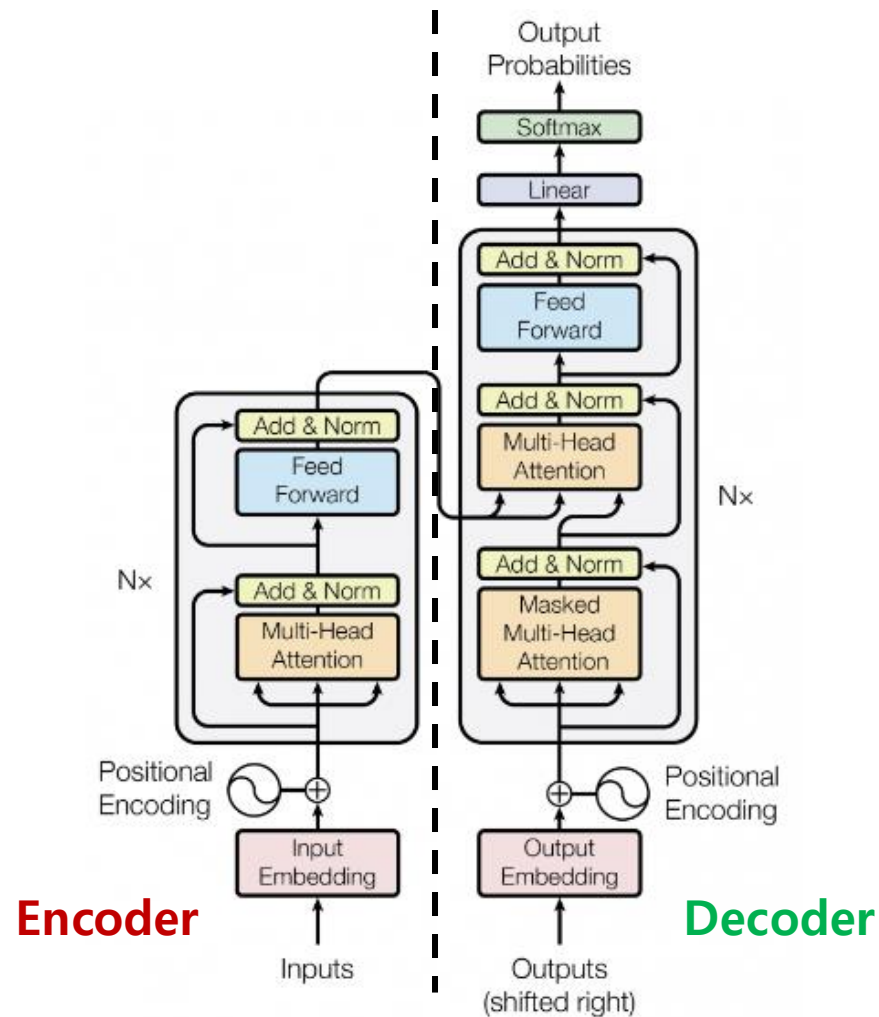
“어제 카페 갔었어 거기 사람 많더라”

- 'cafe'라는 단어를 생성하기 위해 유사도가 높은 '카페', '거기' 등의 단어에 Attention 하여 단어를 생성한다.



Recap: Attention

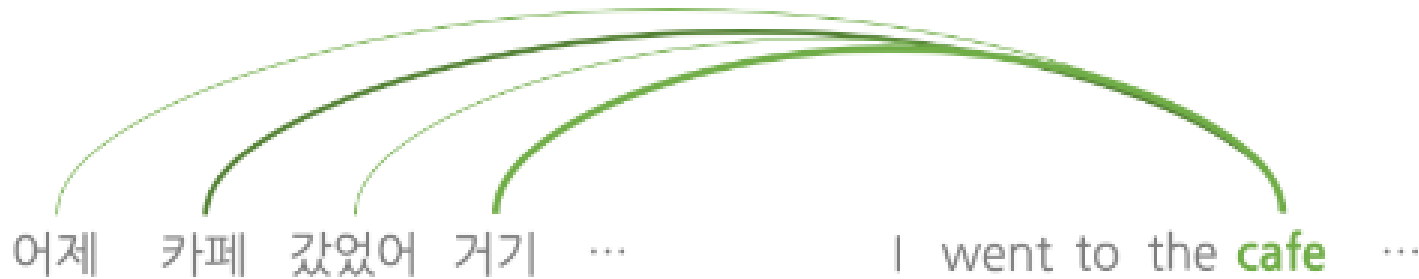




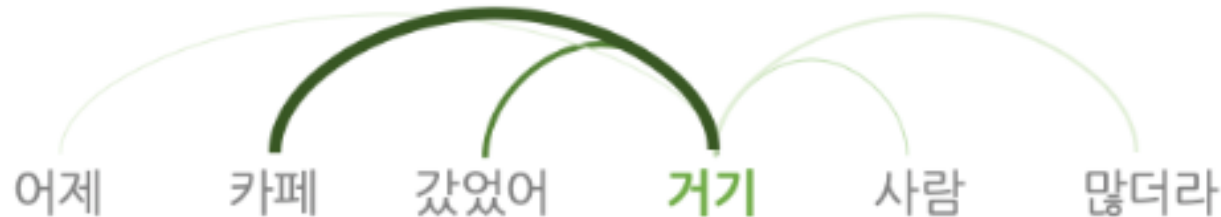
- Encoder-Decoder 구조의 architecture
- Input sentence 내에 존재하는 단어들 간의 상호작용만으로 표현을 학습하는 **Self-Attention** Mechanism을 사용하는 모델

Transformer – Self-Attention

- Attention은 input sequence에서 유사성이 높은 단어를 참조하는 과정

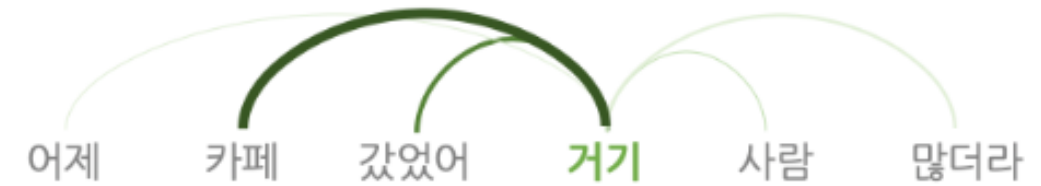


- **Self-Attention**은 input sequence 문장 내에서 각 단어 간 유사성(관계)을 파악하는 과정
- 사람은 문맥 상 '거기'는 '카페'를 의미한다는 것을 쉽게 파악할 수 있음

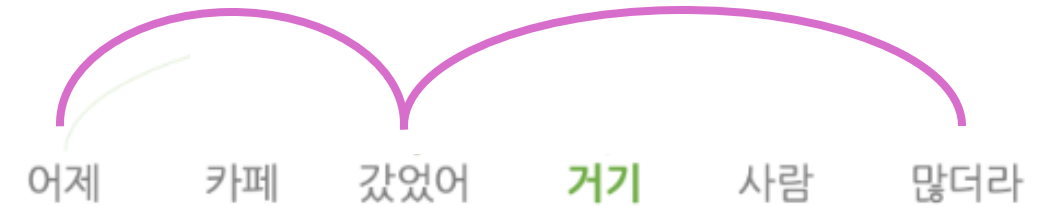


Transformer – Multi-head Attention

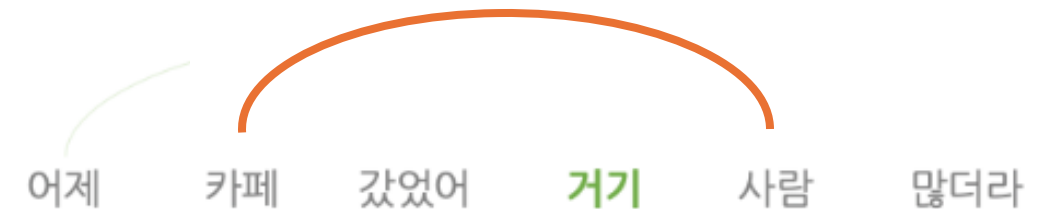
- Attention Head 1 – 단어의 문맥적 관계에 Attention



- Attention Head 2 – 단어의 시제에 Attention



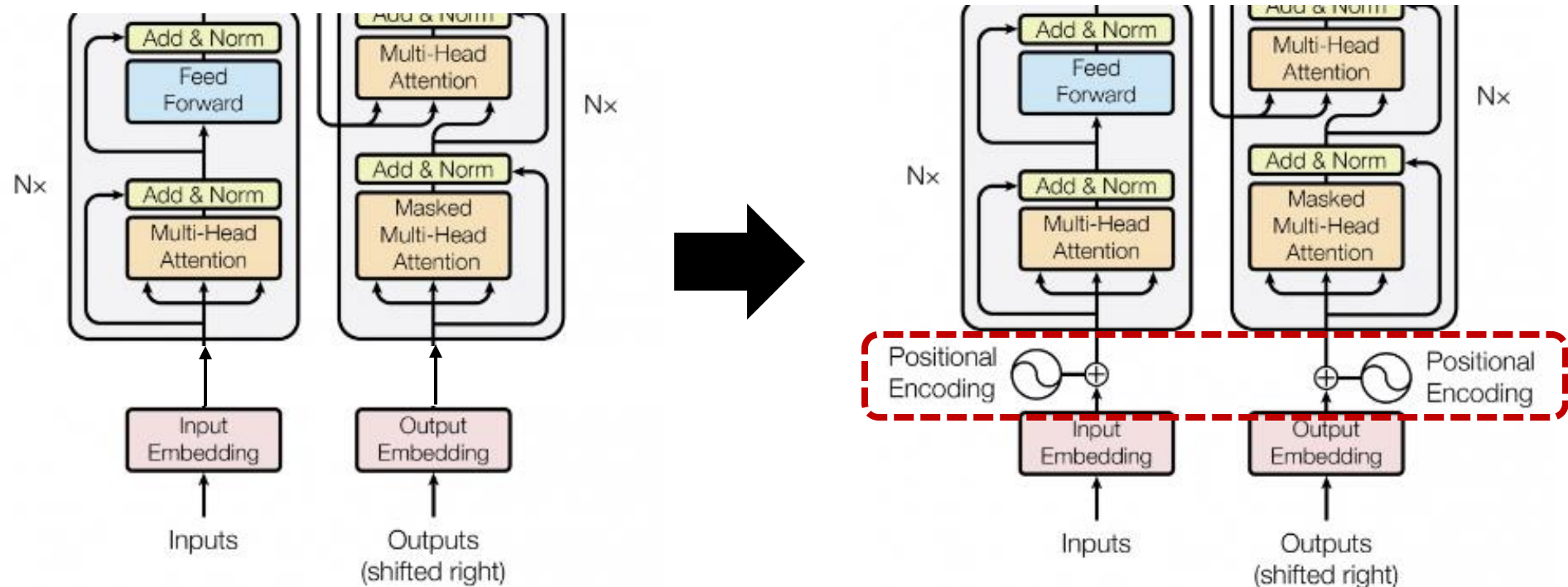
- Attention Head 3 – 명사에 Attention



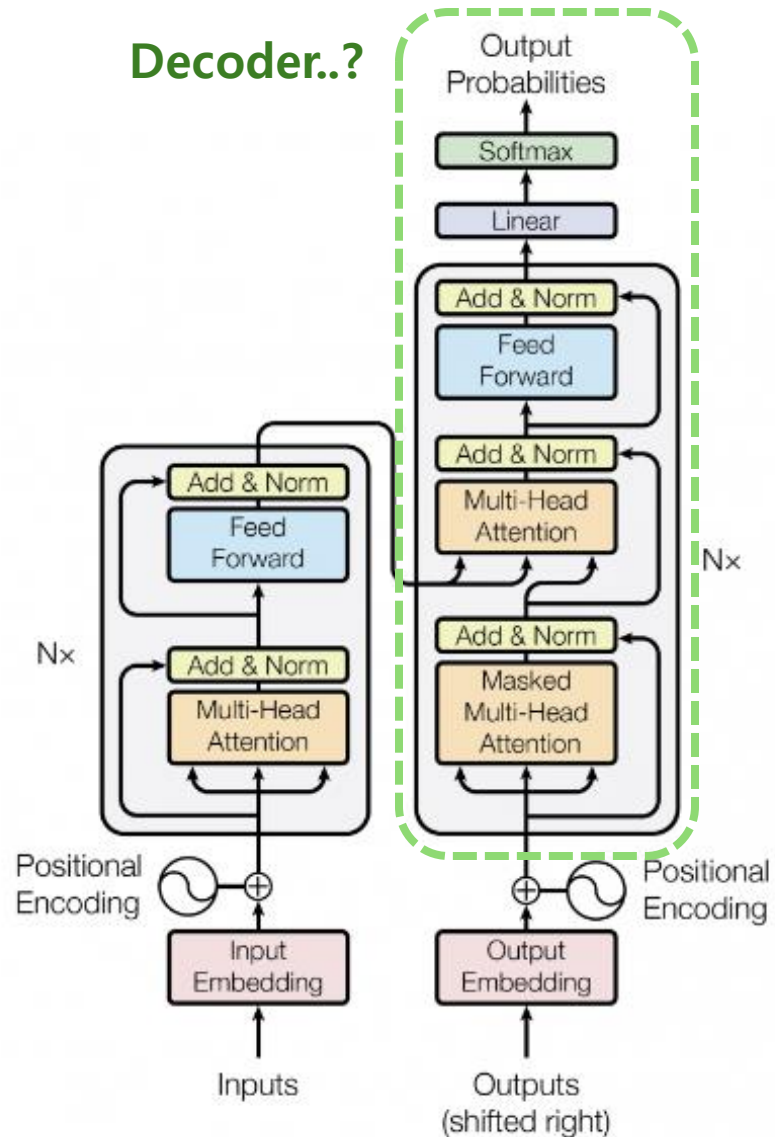
- 즉, 문장 내 **다양한 측면의 정보를** 파악하기 위해 Self-Attention 을 여러 번 수행하는 것

Transformer – Positional Encoding

- 하지만, Self-Attention mechanism 자체는 input sequence의 순서를 고려하지 않음
→ 이를 해결하기 위해, 각 단어의 위치 정보를 담은 벡터(positional encoding)를 입력에 더함



Transformer – Decoder



- Transformer의 **Encoder**를 통해 Natural Language Understanding 능력을 학습함
- 기존 Encoder-Decoder 구조에선, Decoder를 통해 생성을 담당
→ Transformer의 Decoder는 Encoder의 natural language 이해 능력을 어떻게 활용할까?

Transformer – Masked Attention

Masked Attention

the	cake	was	sour
the	cake	was	sour
the	cake	was	sour
the	cake	was	sour

+

0	-inf	-inf	-inf
0	0	-inf	-inf
0	0	0	-inf
0	0	0	0

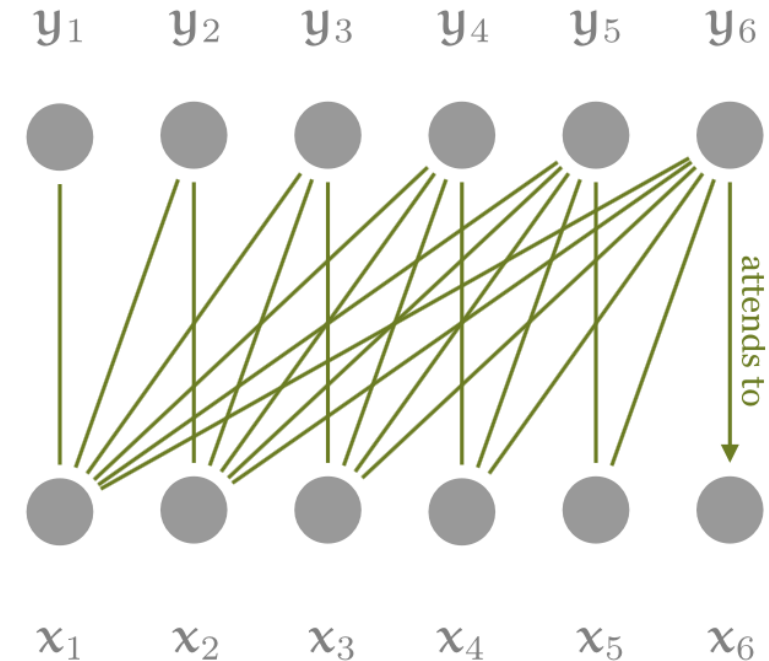
=

the	-inf	-inf	-inf
the	cake	-inf	-inf
the	cake	was	-inf
the	cake	was	sour

Attention Matrix

Masked Matrix

Resultant Matrix

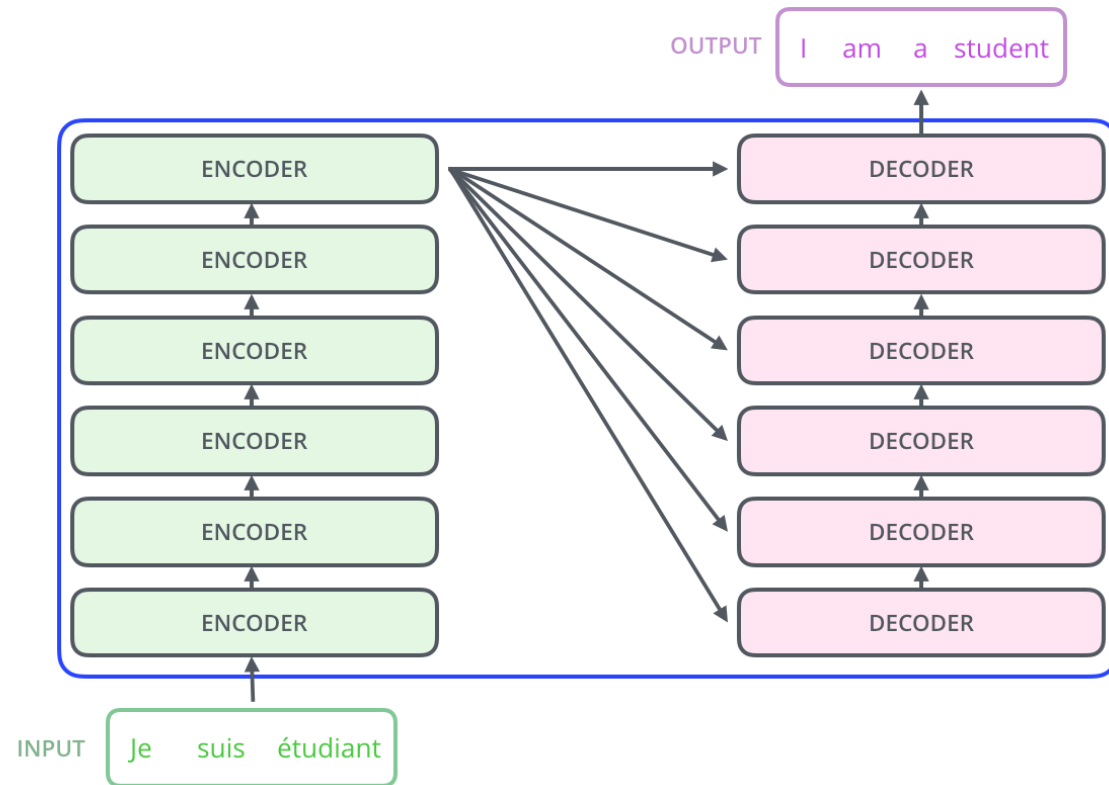
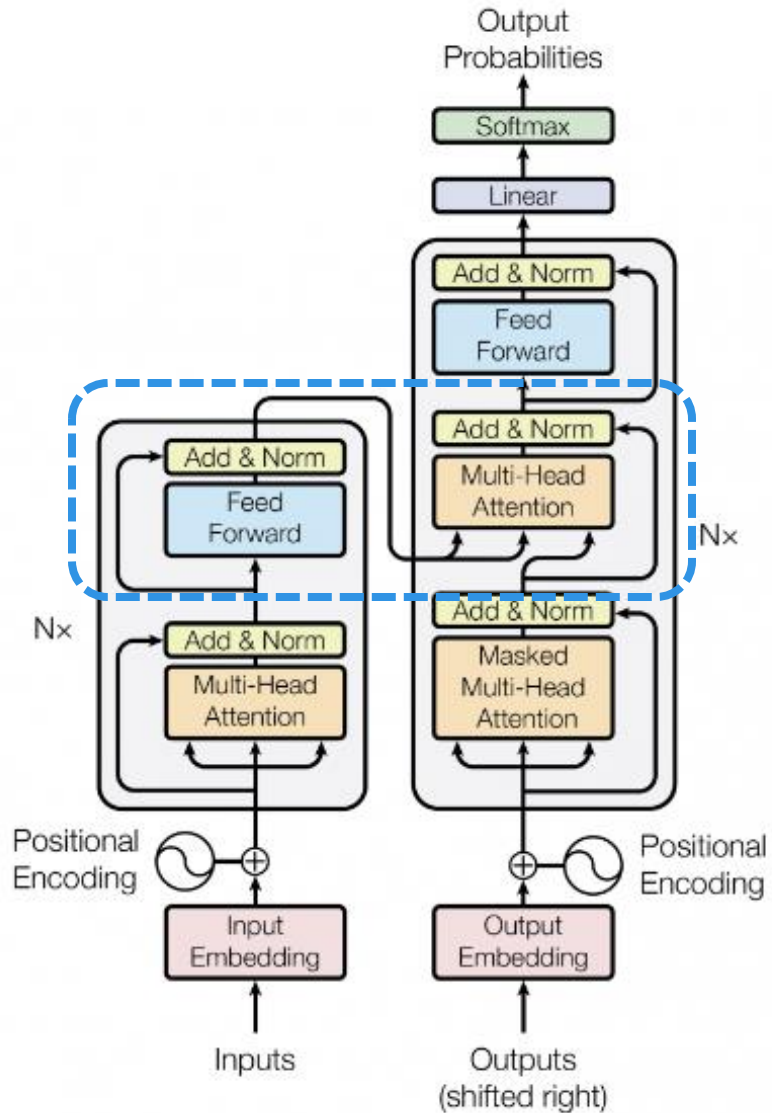


- “The cake was sour” 이라는 문장을 학습한다고 할 때, 각 단어 생성 시 미래의 단어는 볼 수 없도록 했음
(ex. “was” 생성 시 “the”, “cake” 만 input으로 받고, “sour”는 받지 않음)

→ Next Token Prediction

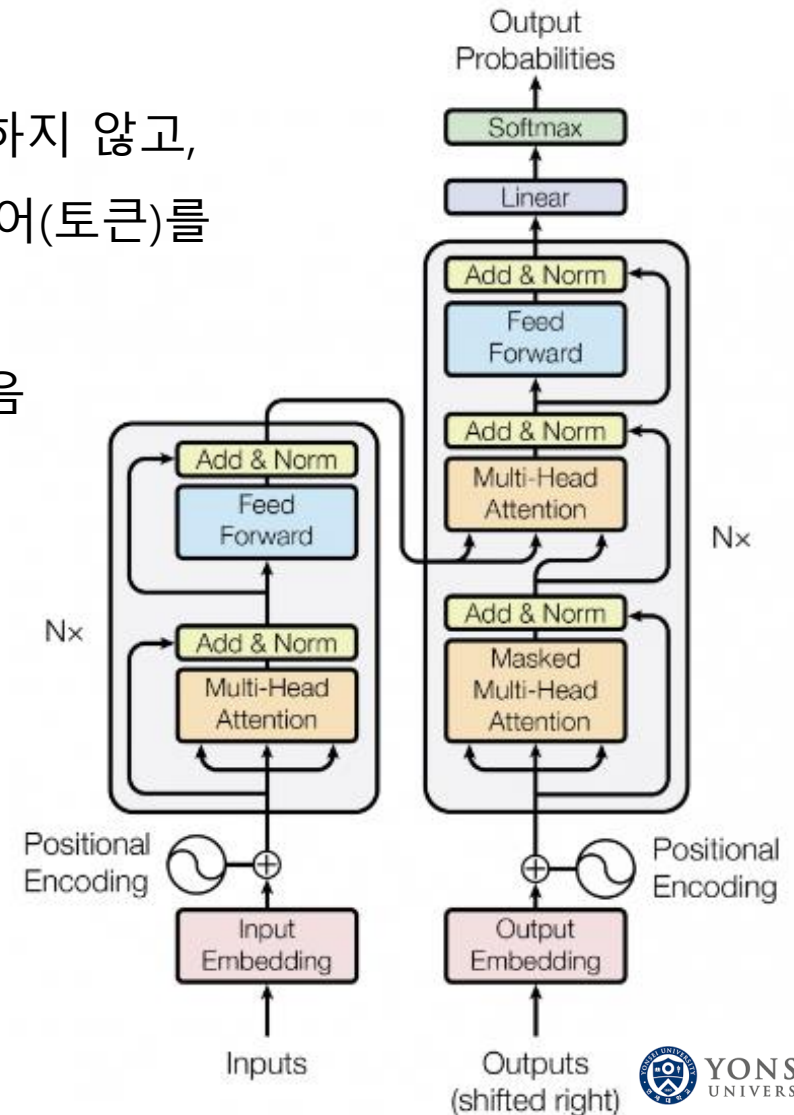
Transformer – Decoder

- Next Token Prediction 과정 중에, encoder에서 학습한 단어의 representation을 전달받음



Transformer

- **Attention mechanism**으로 **Long-range dependency**
(멀리 있는 단어에도 attention하여 참조할 수 있다.)
 - **Self-Attention mechanism**을 통해 RNN처럼 순차적으로 입력을 처리하지 않고,
positional encoding을 통해 순서를 표현해 input sequence의 모든 단어(토큰)를
병렬적으로 연산
 - **Multi-head Attention**을 통해 문장 내에서 **더 많은 정보**를 얻을 수 있음
 - **Next Token Prediction**을 통해 natural language **generation**을 학습
 - Decoder에서 **Encoder의 representation**을 참고해 natural language generation에도 적용
- 자연어에 대한 강력한 이해를 바탕으로 다양한 Tasks에 적용이 가능



Hugging Face 이용 Transformer 실습

Hugging Face Transformers 란?

- Transformer 계열의 여러 모델을 쉽게 사용할 수 있도록 여러 기능을 제공하는 라이브러리

실습 내용

- HuggingFace Transformers 라이브러리 설치 및 사용법 익히기
- 여러 NLP Task를 직접 수행해보기



Hugging Face

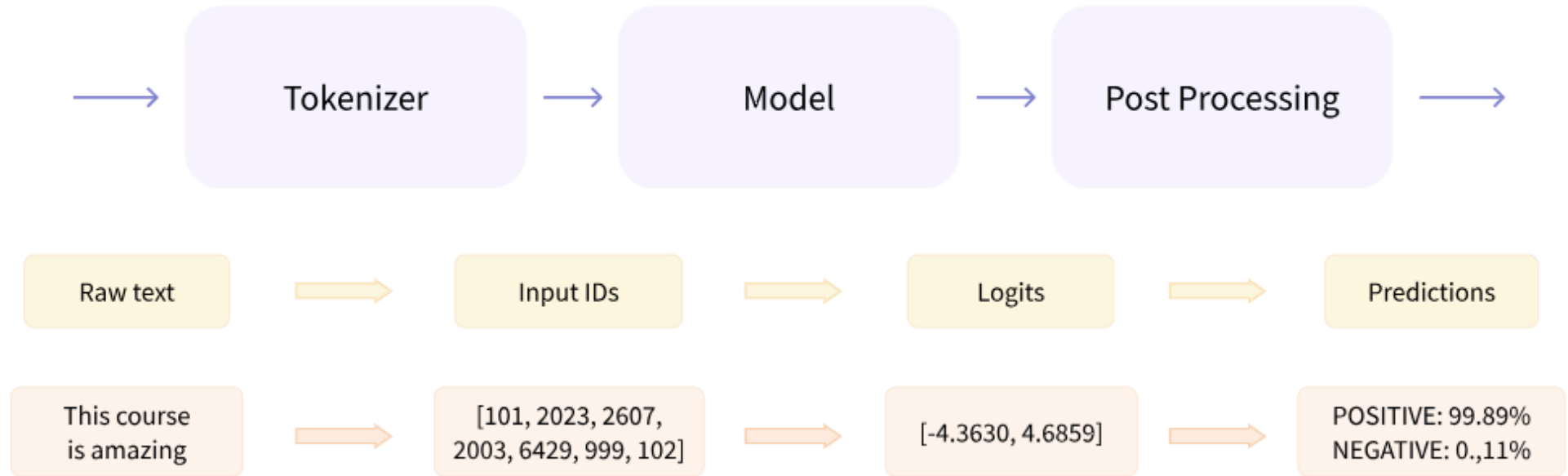
Hugging Face란?

- **Hugging Face**는 대규모 사전학습 모델(Pretrained Models)을 누구나 쉽게 사용할 수 있도록 API, Library, dataset, model hub, spaces, community 등을 제공하는 플랫폼
- 주소
 - Main homepage: <https://huggingface.co/>



Hugging Face

Hugging Face - Pipeline



Pipeline API

Hugging Face - Pipeline

Pipeline

<Parameters>

- Task
- Model
- Device

...



Pipeline API

Natural Language Understanding & BERT

Natural Language Understanding (NLU)

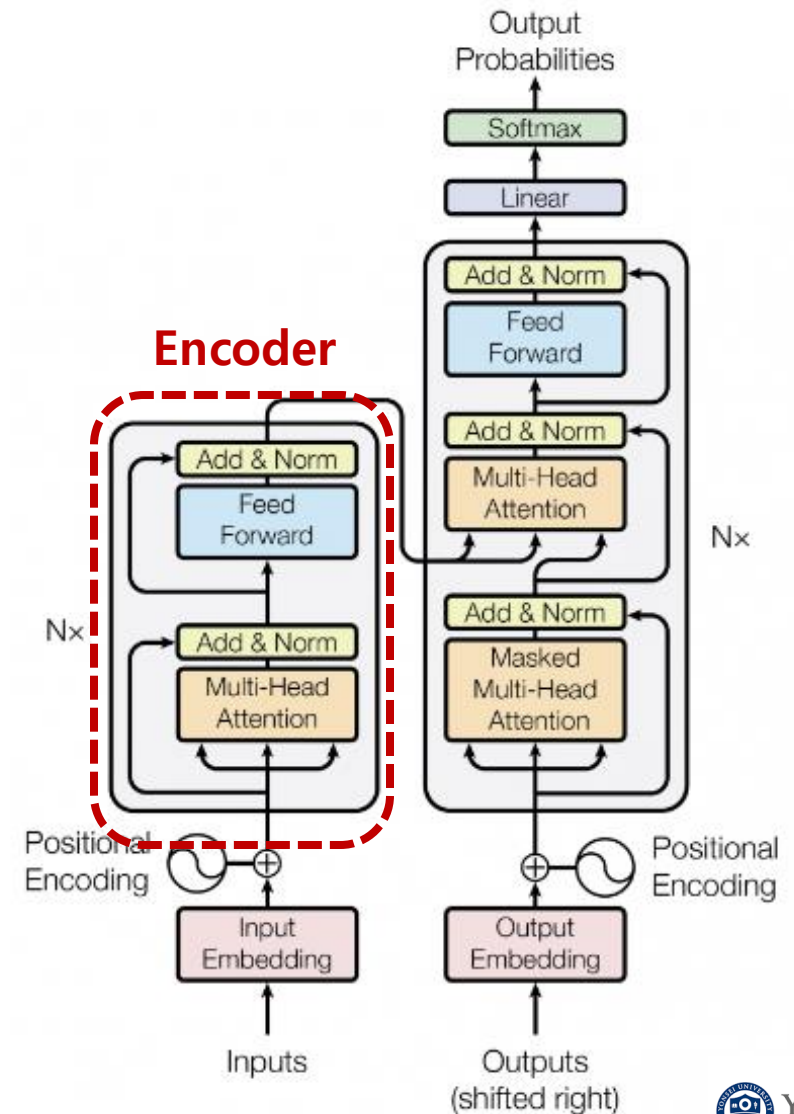
- 자연어 처리 (NLP) = 자연어를 이해하는 과정 (NLU) + 자연어를 생성하는 과정 (NLG)

과제	벤치마크	예제	
감정 분석 Sentiment Analysis	SST, IMDB, NSMC	입력	문장: 신만이 이 영화를 용서할수 있다
		출력	긍정 · 부정
유사도 예측 Similarity Prediction	STS, MRPC, QQP, PAWS-X	입력	문장1: 파이썬과 자바 중 뭐부터 배워야 하나요? 문장2: Java나 Python 중 하나를 배워야 한다면, 뭐부터 시작해야 할까요?
		출력	유사 · 비유사
자연어 추론 Natural Language Inference	SNLI, MNLI, XNLI, aNLI	입력	전제: 회색 개가 숲에서 쓰러진 나무를 핥고 있다. 가설: 개가 밖에 있다.
		출력	참 · 거짓 · 모름
언어적 용인 가능성 Linguistic Acceptability	CoLA	입력	문장: 그는 한 번도 프랑스에 갔다.
		출력	가능 · 불가능
기계 독해 Reading Comprehension	SQuAD, RACE, MS MARCO, KorQuAD	입력	지문: 시카고 대학의 학자들은 경제학, 사회학, 법학 분석, 문학 비평, 신학, 행동주의 정치학 등 다양한 학문 발전에 중요한 역할을 담당해왔다. [...] 이 대학은 미국 최대 대학 출판사인 시카고 대학 프레스(University Press of Chicago Press)의 본거지이기도 하다. 오는 2020년 완공될 버락 오바마 대통령 센터에는 오바마 대통령 도서관과 오바마 재단 사무소가 세워질 예정이다. 질문: 버락 오바마 대통령 센터의 완공 예정연도는?
		출력	2020년
의도 분류 Intent Classification	ATIS, SNIPS, AskUbuntu	입력	질문: 현재 위치 주변에 있는 맛집 두 곳만 알려줘
		출력	장소 검색 (식당, 가까운, 평점 좋은, 2)

Transformer – Encoder

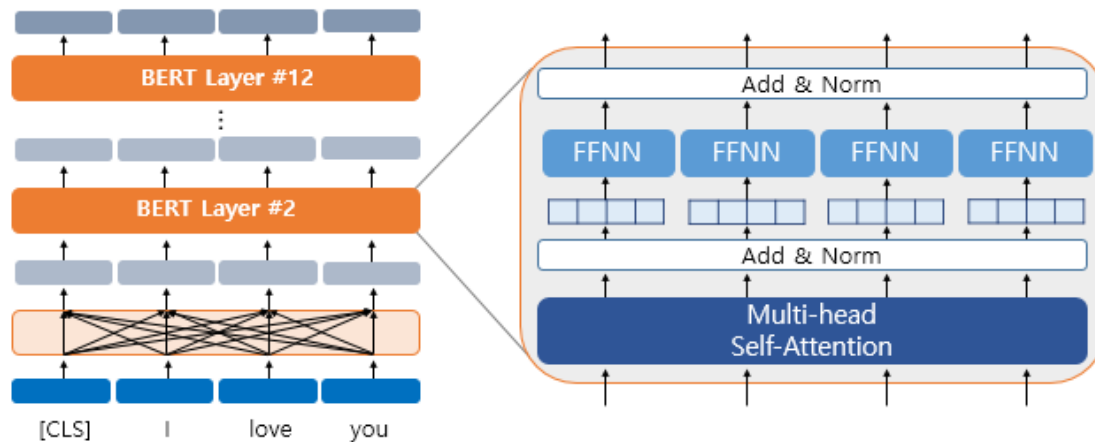
- Transformer의 **Encoder**가 **Natural Language Understanding**에 적합한 architecture
- 즉, Natural Language를 이해해서 의미를 추출하고 context를 반영한 표현을 만드는 데에 최적화 되어있다
- 이러한 Encoding 능력을 극대화해보자!

→ BERT

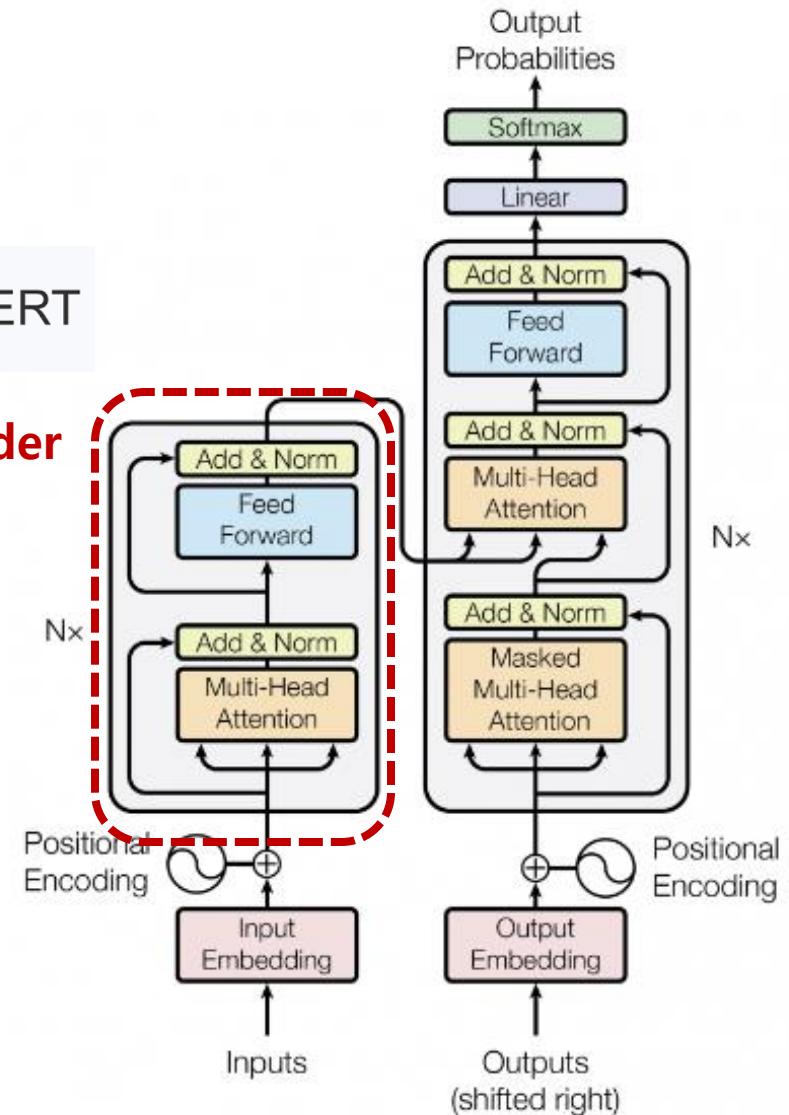


BERT

- 2018년 구글이 공개한 **Pre-trained** 언어 모델
- Transformer 구조에서 **Encoder**만 사용한 형태
- NLU 태스크에 적합한 형태



Encoder

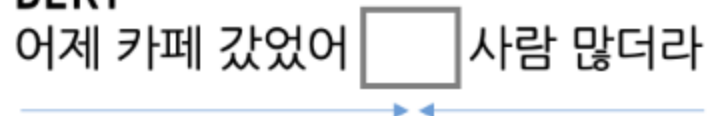


BERT

- Input sequence를 양방향(Bidirectional)으로 학습하여 문맥 이해에 특화

BERT

어제 카페 갔었어 사람 많더라



BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Jacob Devlin Ming-Wei Chang Kenton Lee Kristina Toutanova

Google AI Language

{jacobdevlin, mingweichang, kentonl, kristout}@google.com

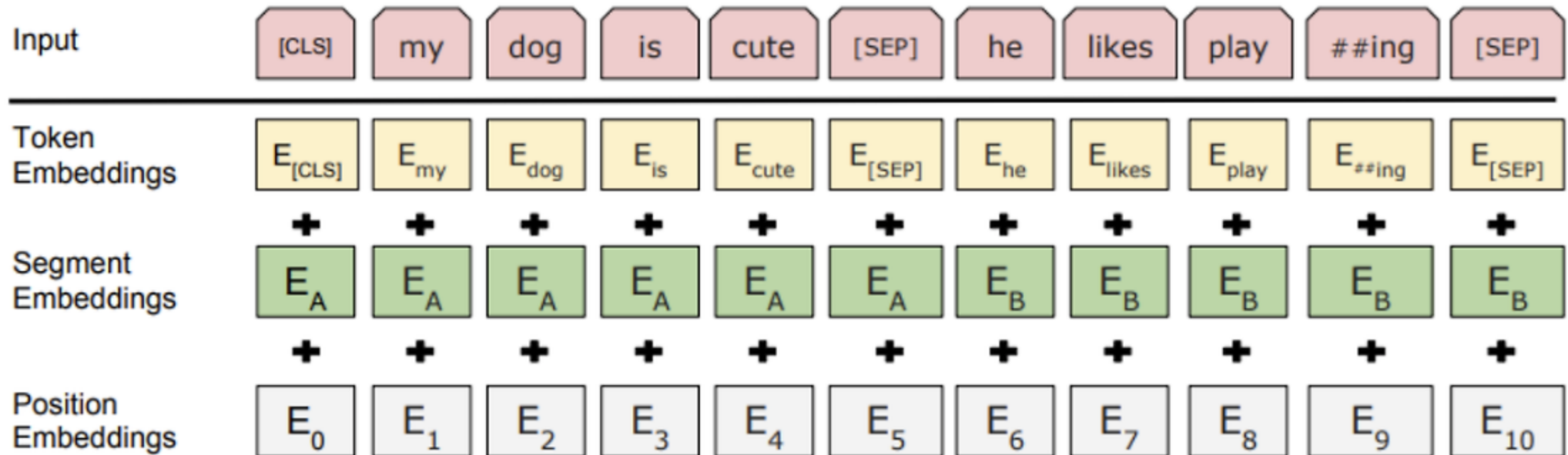
Abstract

We introduce a new language representation model called **BERT**, which stands for **Bidirectional Encoder Representations from**

There are two existing strategies for applying pre-trained language representations to downstream tasks: *feature-based* and *fine-tuning*. The feature-based approach, such as ELMo (Peters

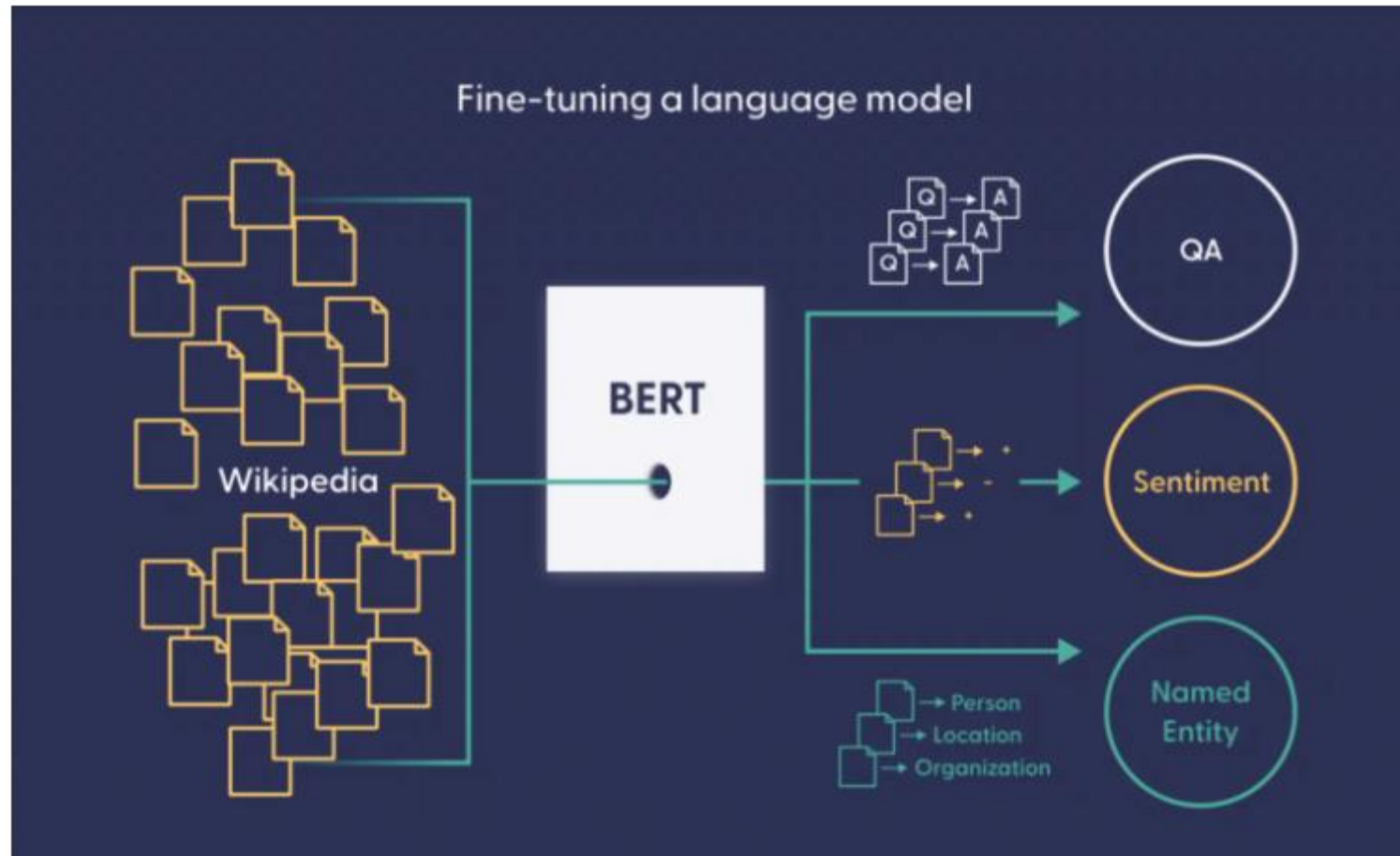
BERT (Data Preprocessing)

- Input sequence를 Token embedding + Segment embedding + Position Embedding으로 표현



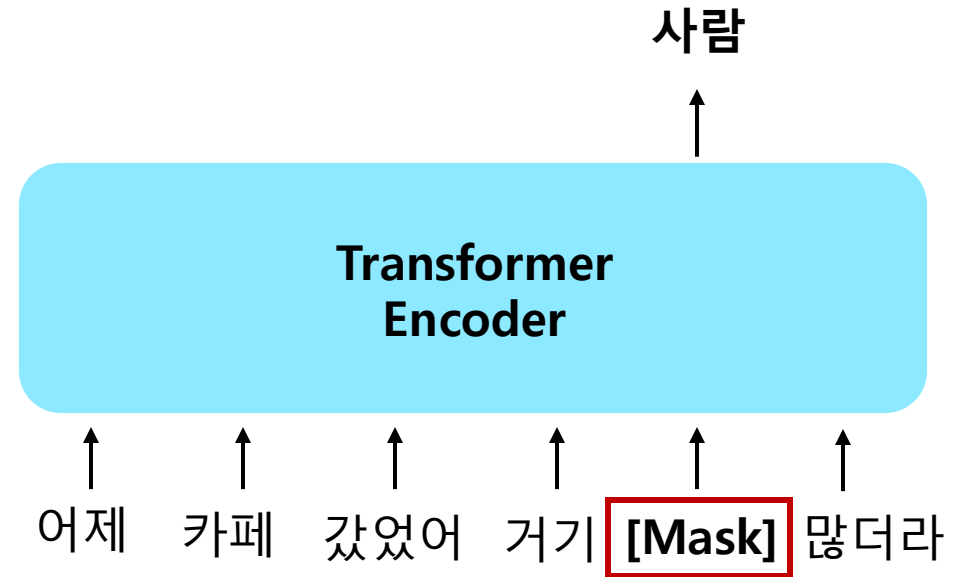
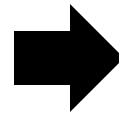
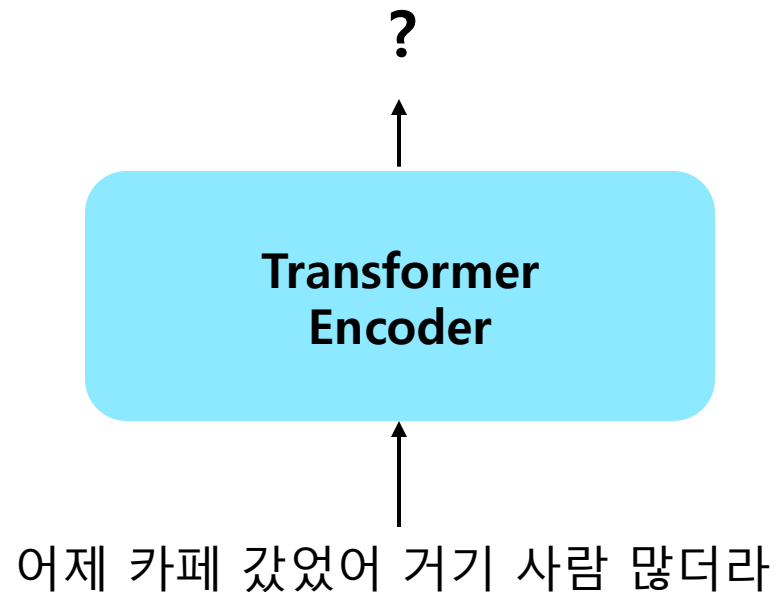
BERT

- Wikipedia + Book (약 33억 개 데이터 학습) → 어떻게 가공했을까? (e.g. labeling)



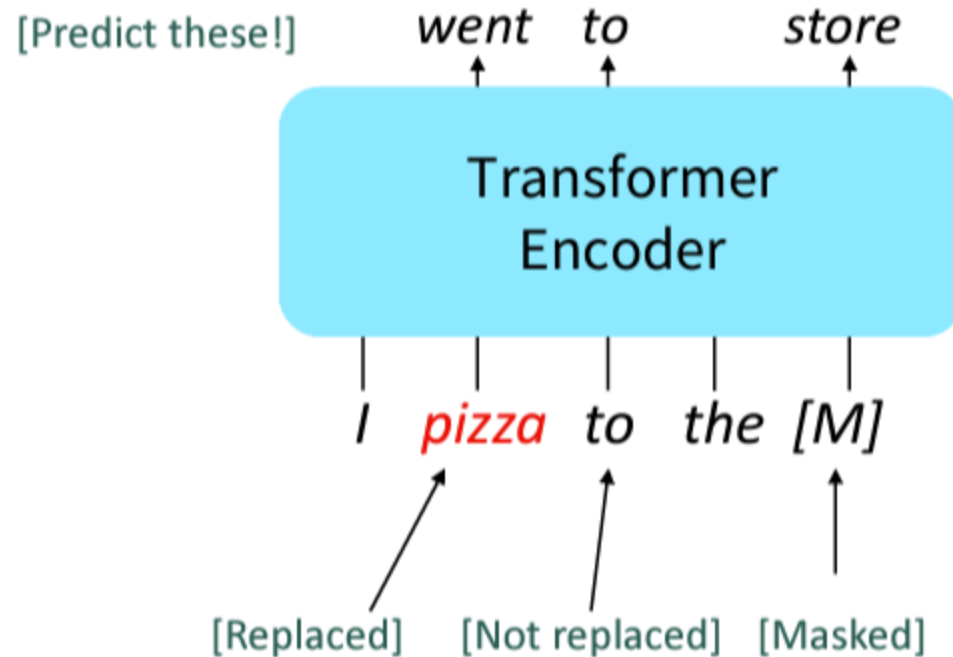
BERT (Masked Language Model)

- Input sequence의 일부 단어를 random하게 **Masking** 하고 해당 단어를 예측하도록 모델을 학습
- 해당 과정에서 문맥을 파악하는 능력이 학습됨



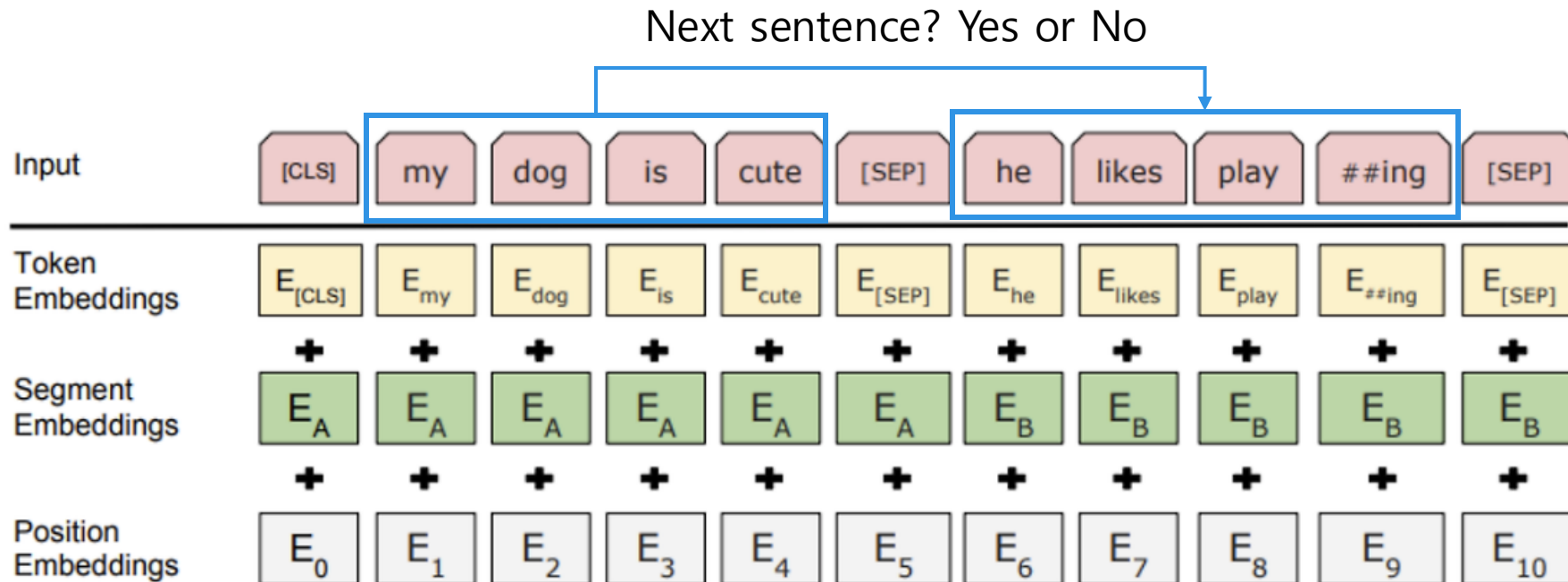
BERT (Masked Language Model)

- 또는 특정 단어를 random하게 다른 단어로 replace한 뒤 해당 단어를 예측하도록 학습
- 이러한 학습 과정을 통해 BERT는 문장의 문맥을 파악하는 능력을 학습하게 됨



BERT (Next Sentence Prediction)

- 주어진 두 문장이 이어지는 문장인지 판단하는 것을 통해 학습
 - 이어진다 (IsNext: 0), 이어지지 않는다 (IsNotNext: 1)
- 즉, BERT는 두 문장 간의 관계를 파악하는 능력을 학습



BERT 실습 (Tokenizer)

- BERT의 Tokenizing 방식을 실습
- Tokenizing - 입력된 텍스트를 모델에서 처리할 수 있는 데이터(숫자)로 변환하는 과정
- Transformers 패키지를 사용하여 사용하고자 하는 모델의 Tokenizer를 load
- 사용하는 Tokenizer는 항상 맵핑 관계여야 함.
 - pre-train 시 사용했던 tokenizer는 '사과'를 10번으로 인코딩
 - 만약 모델 사용 시 사용하는 tokenize가 '사과를 25번으로 인코딩 한다면?
→ '사과'라는 단어의 텍스트를 모델이 이해할 수 없다
- **Sub word Tokenization** 방식 (WordPiece)
→ 어떤 sub word로 나뉘었을 때, 언어모델의 예측 성능이 늘어나는가?
(unaffordable -> ["un", "##afford", "##able"] or ["un", "##affordable"])
(## → 중간 / 끝의 subword라는 것 알리기)

BERT의 토크나이저 : WordPiece

```
import pandas as pd
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
```

BERT 실습 (Masked LM)

- BERT가 Masking된 토큰을 예측하는 방식을 실습

실제로 [MASK]를 예측해보자

```
from transformers import FillMaskPipeline  
fill_mask = FillMaskPipeline(model=model, tokenizer=tokenizer)  
  
fill_mask('Samsung is a [MASK] company in korea.')
```

BERT 실습 (Next Sentence Prediction)

- BERT가 두 개의 문장이 이어지는 문장인지 판단하는 방식을 실습

✓ Next Sentence Prediction 알아보기

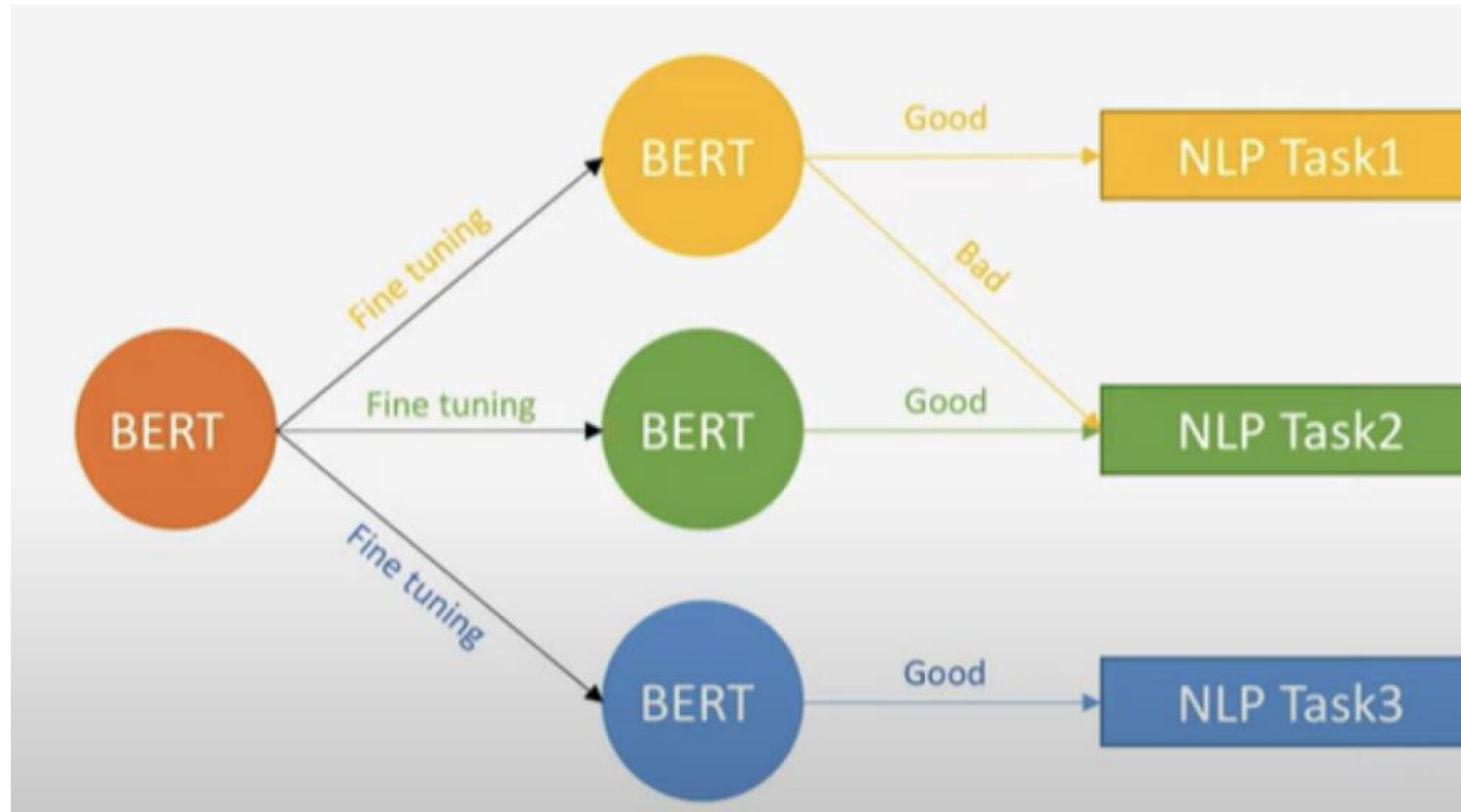
```
[33] from transformers import BertForNextSentencePrediction  
      from transformers import AutoTokenizer  
      import torch  
      import torch.nn.functional as F
```

```
[34] model = BertForNextSentencePrediction.from_pretrained('bert-base-uncased')  
      tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
```

```
[41] prompt = "I love Samsung"  
      next_sentence = "Samsung is a good company in korea"
```

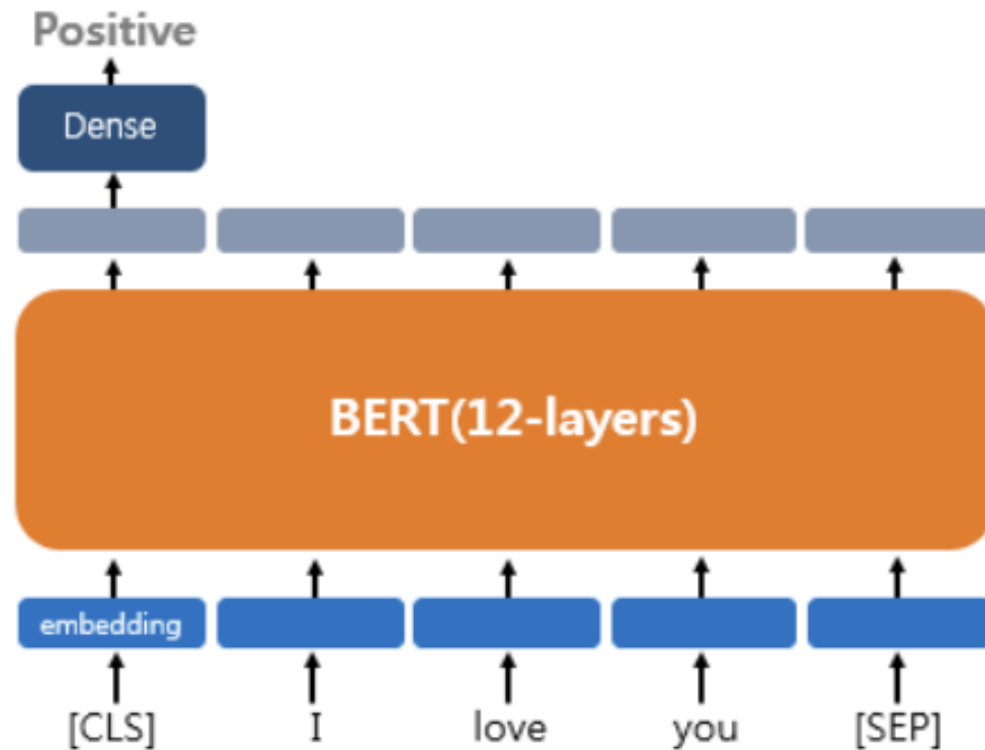
BERT 실습 (Fine-tuning)

- 앞서 설명한 방법을 통해 Pre-train 된 모델이 존재함
- 이를 각 task에 맞게 fine-tuning하여 사용



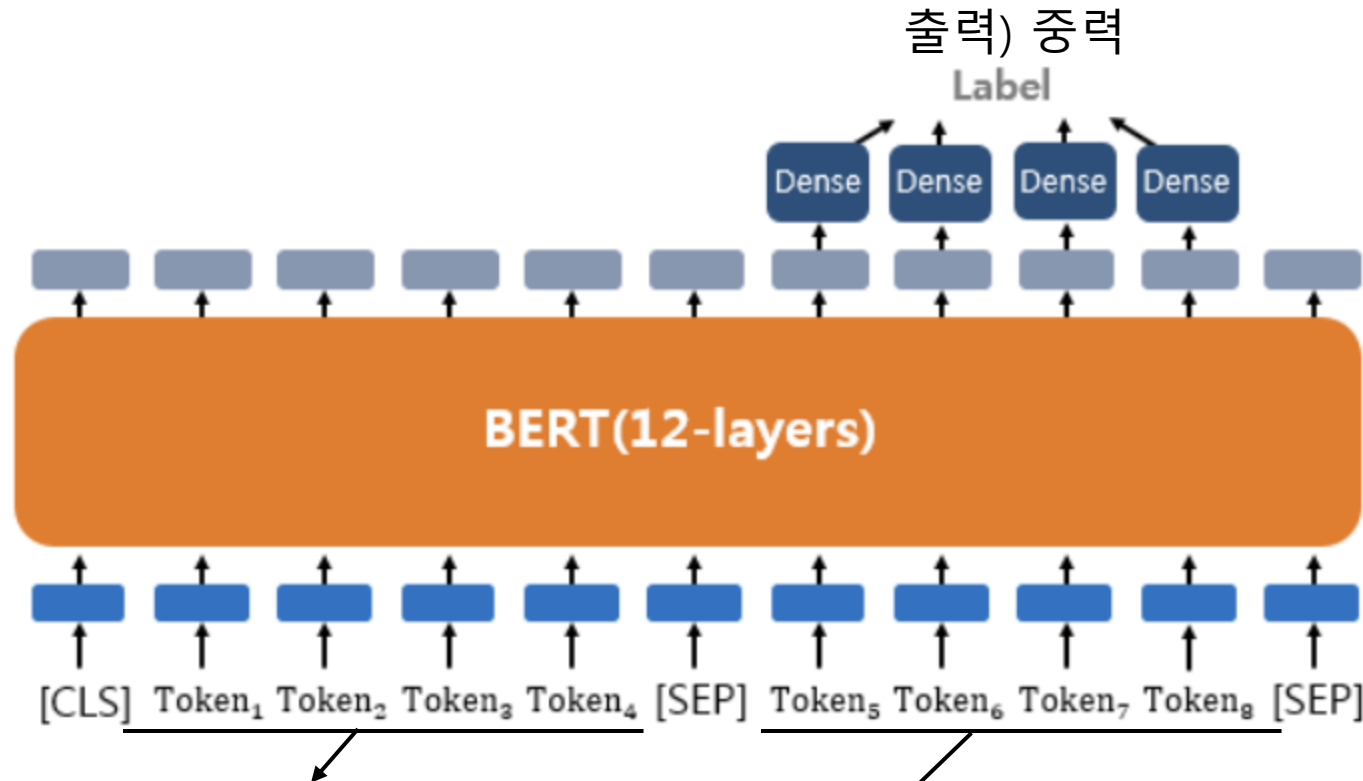
BERT 실습 (Fine-tuning)

- 문장 분류를 위한 Fine-tuning 예시
- [CLS] 토큰의 출력층에 Dense Layer를 추가해서 분류에 대한 예측을 할 수 있게 됨



BERT 실습 (Fine-tuning)

- QA(Question Answering)를 위한 Fine-tuning 예시



입력 1 - 질문) 강우가 떨어지도록 영향을 주는 것은?

입력 2 - 문서) 기상학에서 강우는 대기 수증기가 응결되어 중력의 영향을 받고 떨어지는 것을 의미합니다.

BERT 실습 (Fine-tuning)

- BERT 기반 네이버 영화 리뷰 감성 분석 모델 Fine-tuning

	id	document	label
0	9976970	아 더빙.. 진짜 짜증나네요 목소리	0
1	3819312	흠...포스터보고 초딩영화줄....오버연기조차 가볍지 않구나	1
2	10265843	너무재밌었다그래서보는것을추천한다	0
3	9045019	교도소 이야기구먼 ..솔직히 재미는 없다..평점 조정	0
4	6483659	사이몬페그의 익살스런 연기가 돋보였던 영화!스파이더맨에서 늙어보이기만 했던 커스틴 ...	1
5	5403919	막 걸음마 떼는 3세부터 초등학교 1학년생인 8살용영화.ㅋㅋㅋ...별반개도 아까움.	0
6	7797314	원작의 긴장감을 제대로 살려내지못했다.	0
7	9443947	별 반개도 아깝다 욕나온다 이응경 길용우 연기생활이몇년인지..정말 발로해도 그것보단...	0
8	7156791	액션이 없는데도 재미 있는 몇안되는 영화	1
9	5912145	왜케 평점이 낮은건데? 꽤 볼만한데.. 할리우드식 화려함에만 너무 길들여져 있나?	1

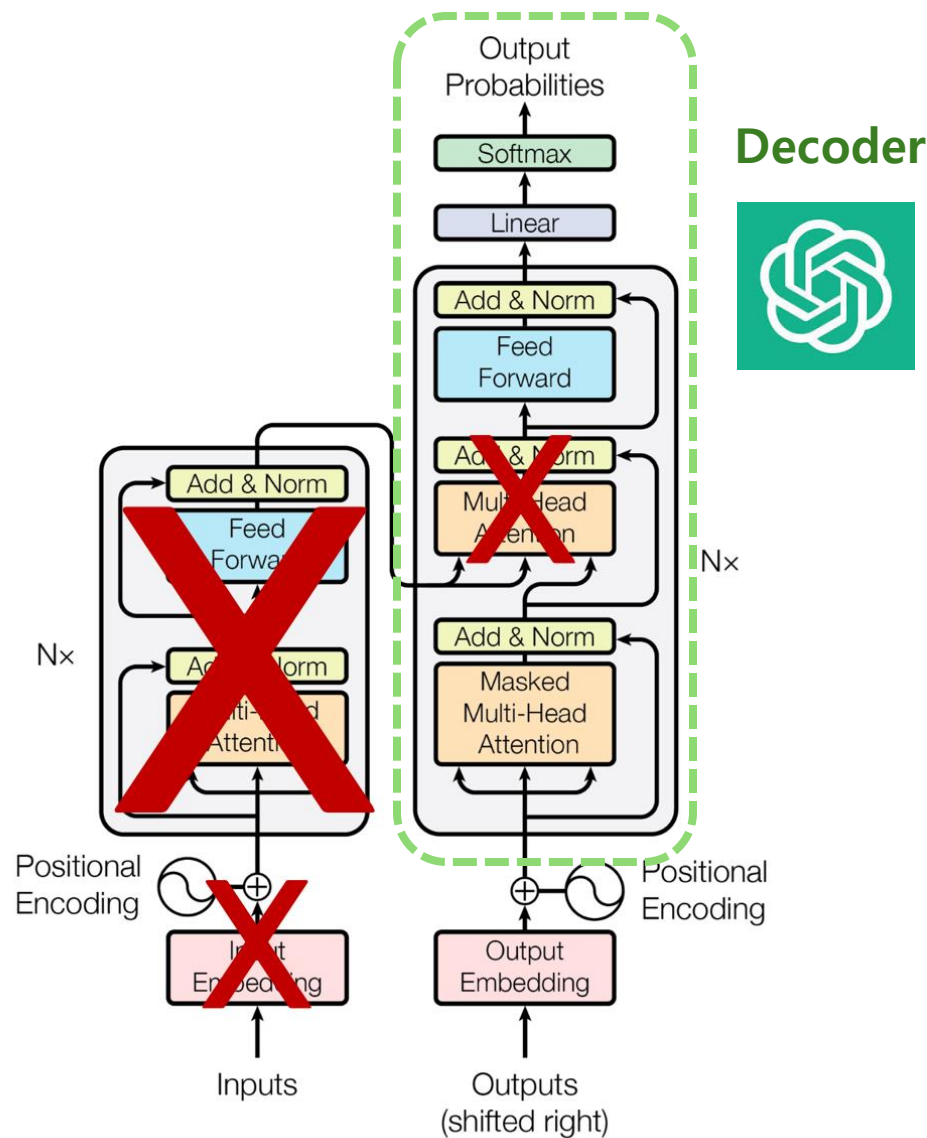
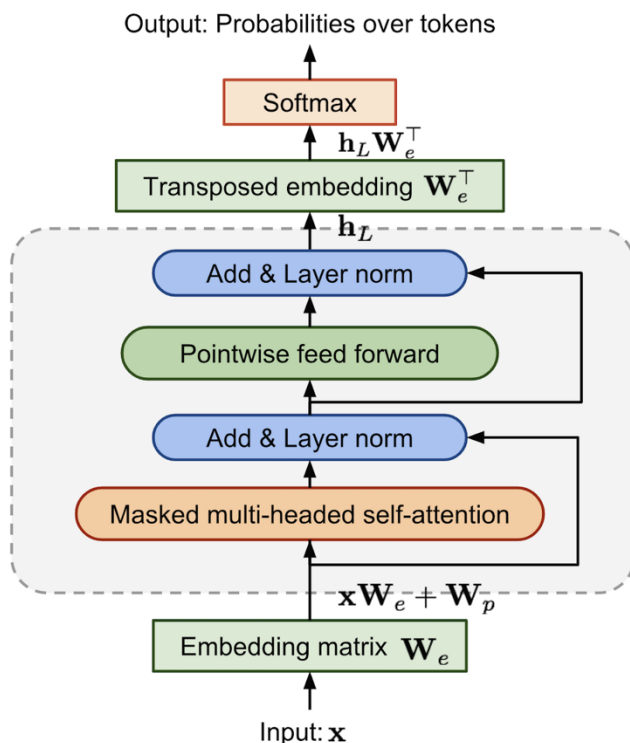
BERT 실습 (Fine-tuning)

- BERT 기반 한국어 감정 분류 모델 Fine-tuning
- klue-bert (한국어를 학습한 bert 모델)

	Sentence	Emotion
0	언니 동생으로 부르는게 맞는 일인가요..??	공포
1	그냥 내 느낌일뿐겠지?	공포
2	아직너무초기라서 그런거죠?	공포
3	유치원버스 사고 났다던데	공포

Natural Language Generation & GPT

- 2018년 OpenAI가 공개한 **Pre-trained** 언어 모델
- Transformer 구조에서 **Decoder만 사용한** 형태
- Natural Language Generation에 적합한 architecture

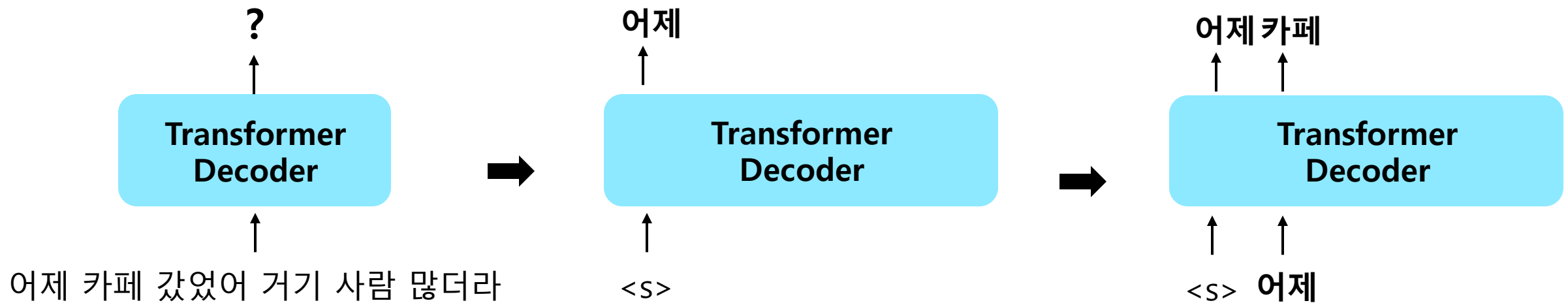


Decoder



GPT (Causal Language Modeling)

- Decoder only architecture이기 때문에, 미래 시점의 정보를 반영할 수 없음
- 따라서, 이전 단어들을 바탕으로 다음 단어를 예측하는 **Next Token Prediction** 방식으로 pre-training을 수행



GPT-1

- **GPT-1**은 Transformer Decoder 기반 Language Model을 **Pre-training** 후 **Fine-tuning**
- 또한, 현재 frontier model인 GPT-4o, o3 / o4 등의 **GPT series의 출발점 역할**을 하는 모델
- 모델의 크기(parameter 수)는 117M로 상대적으로 작은 모델의 크기로 인해 복잡한 언어 이해 및 생성 작업에는 한계를 보임

GPT

어제 카페 갔었어 거기 사람 많더라

Improving Language Understanding by **Generative Pre-Training**

Alec Radford
OpenAI
alec@openai.com

Karthik Narasimhan
OpenAI
karthikn@openai.com

Tim Salimans
OpenAI
tim@openai.com

Ilya Sutskever
OpenAI
ilyasu@openai.com

GPT-2

- GPT-2는 GPT-1에 비해 **parameter 수를 엄청나게 키워**, GPT-1보다 더 긴 문맥을 이해하고, 좀더 일관된 문장을 생성할 수 있게 되었음
- 또한, 더욱 향상된 성능을 바탕으로, fine-tuning 없이 zero-shot setting에서 multitask를 수행할 수 있음을 제안

Language Models are **Unsupervised Multitask Learners**

Alec Radford ^{* 1} Jeffrey Wu ^{* 1} Rewon Child ¹ David Luan ¹ Dario Amodei ^{** 1} Ilya Sutskever ^{** 1}

Abstract

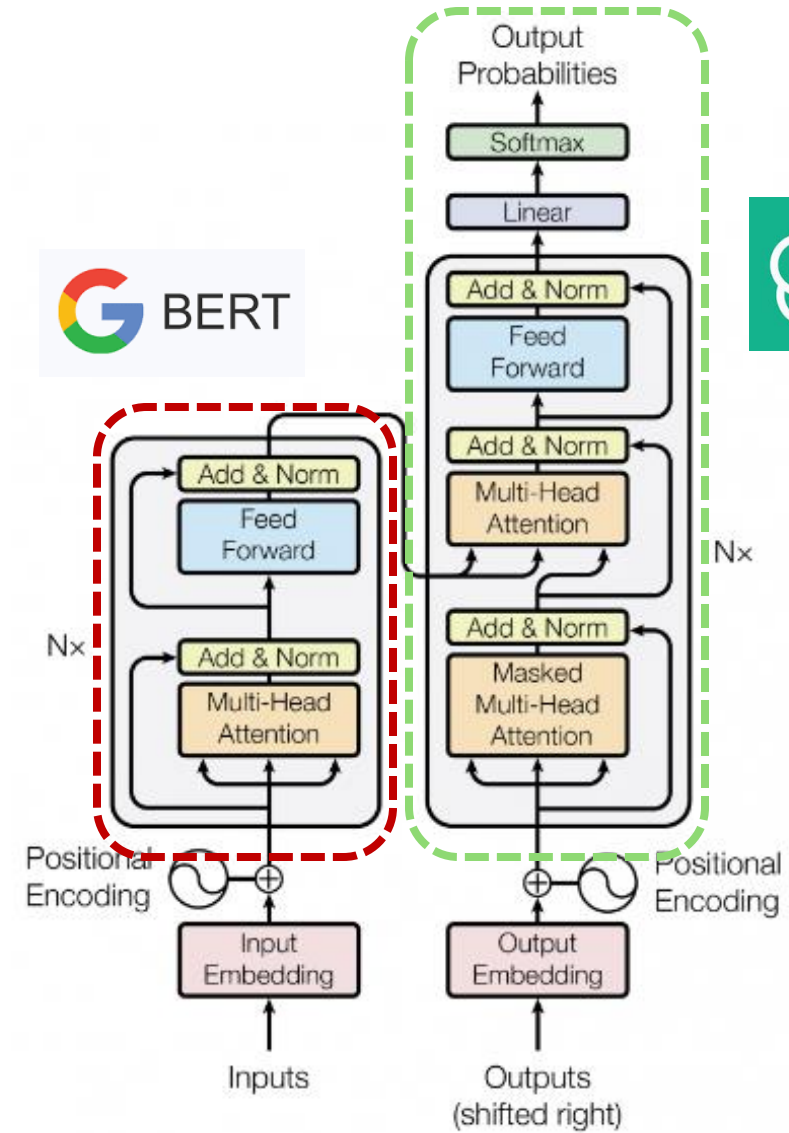
Natural language processing tasks, such as question answering, machine translation, reading comprehension, and summarization, are typically approached with supervised learning on task-specific datasets. We demonstrate that language models begin to learn these tasks without any ex-

competent generalists. We would like to move towards more general systems which can perform many tasks – eventually without the need to manually create and label a training dataset for each one.

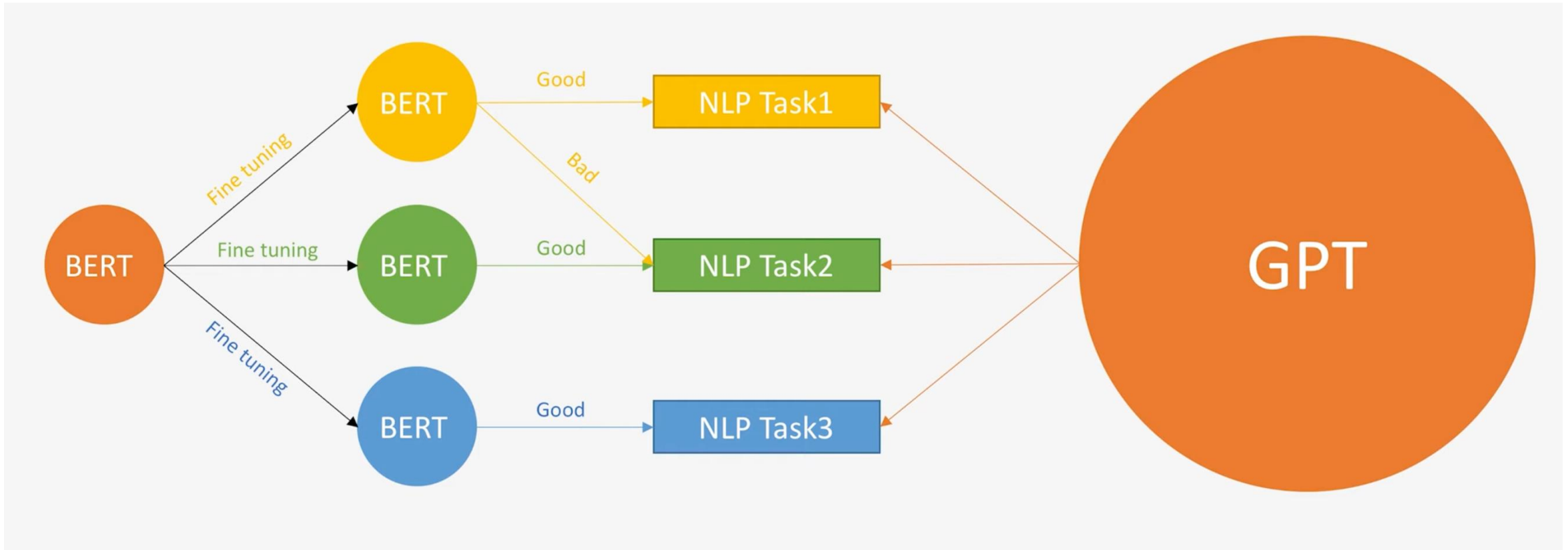
The dominant approach to creating ML systems is to collect a dataset of training examples demonstrating correct behavior for a desired task, train a system to imitate these

BERT vs GPT

- Encoder-only vs Decoder-only
- Bidirectional vs Unidirectional
- Masked Token 복원 vs Next Token 생성
- Fine-tuning 필요 vs Fine-tuning 없이 Multi-task 가능



BERT vs GPT



- 즉, GPT는 task 별로 각기 다른 fine-tuning이 필요하지 않은 General한 모델
→ GPT의 Next Token Prediction이라는 학습 방식 때문

BERT vs GPT

- GPT는 BERT와 달리 여러 문장이 연결된 하나의 긴 sequence에 대해 next token prediction 학습
- 즉, **Input**이자 **Task**가 주어졌을 때 **Output의 확률을 예측**하는 방식이라 할 수 있음

$$p(output|input) \longrightarrow p(output|input, task)$$

번역 task 예시

“**Brevet Sans Garantie Du Gouvernement**”, translated to English: “**Patented without government warranty**”.

문장 내 task가 포함되어 있음

“**Brevet Sans Garantie Du Gouvernement**”, translated to English:

GPT

“**Patented without government warranty**”.

GPT 실습 – 텍스트 생성해보기

- GPT-2로 이어진 context에 대해 텍스트 생성해보는 실습

✓ 텍스트 생성해보기

```
[ ] from transformers import GPT2LMHeadModel, GPT2Tokenizer, pipeline

[ ] # 토큰라이저 및 모델 초기화
    tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
    model = GPT2LMHeadModel.from_pretrained("gpt2")

    # 텍스트 생성 파이프라인 설정
    generator = pipeline("text-generation", model=model, tokenizer=tokenizer)

    # 텍스트 생성 예제
    prompt = "Once upon a time in a faraway land"
    generated_text = generator(prompt, max_length=100, num_return_sequences=1)
    print(generated_text[0]['generated_text'])
```

GPT 실습 – Chat bot 만들어보기

- GPT-2를 이용해, 대화를 주고 받는 간단한 Chat bot을 구현해보기

✓ GPT-2로 챗봇 만들어보기

```
[ ] def chat_with_gpt2(user_input):  
    response = generator(user_input, max_length=50, num_return_sequences=1)  
    return response[0]['generated_text']  
  
# 예시 대화  
conversation = True  
print("Let's talk to GPT-2! If do you want to stop to talk, input the word, 'Stop'")  
  
while conversation == True:  
    user_input = input('You: ')  
    if user_input == 'Stop' or user_input == 'stop':  
        conversation = False  
    else:  
        print('GPT-2: ', chat_with_gpt2(user_input))
```

Decoding Strategies

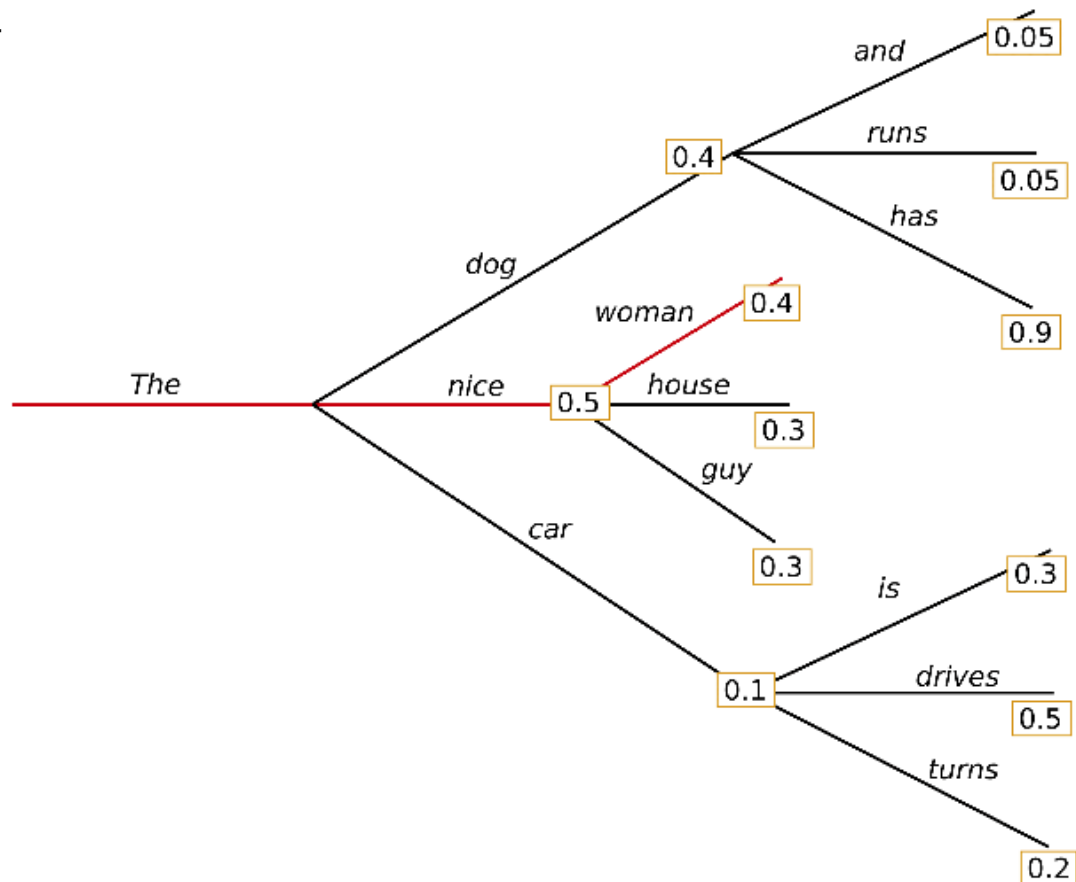
- NLG Model의 결과물은 주어진 context에 대해 **각 token의 조건부 확률 분포**
- 그 중 어떤 token을 활용할 것인가? → Decoding Strategies 필요
- 대표적인 **3가지 Strategies** 존재
 1. **확정적 + Search 기반 전략**
 2. **확률적 + Sampling 기반 전략**
 3. **반복 제어 전략**

Decoding Strategies (1) – 결정적 / Search Based

- Token을 결정할 때, 확률이 높은 token을 선택하기
- 확률이 높은 후보 token을 확정적으로 선택하기 때문에, 결정적이라고 함
 - Input에 대해 항상 같은 output이 나옴
- 대표적으로 두 가지 전략이 존재함

1. Greedy Search

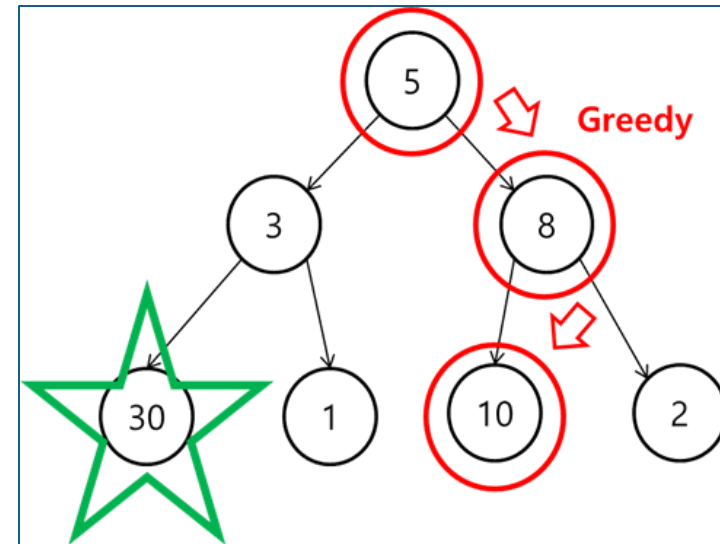
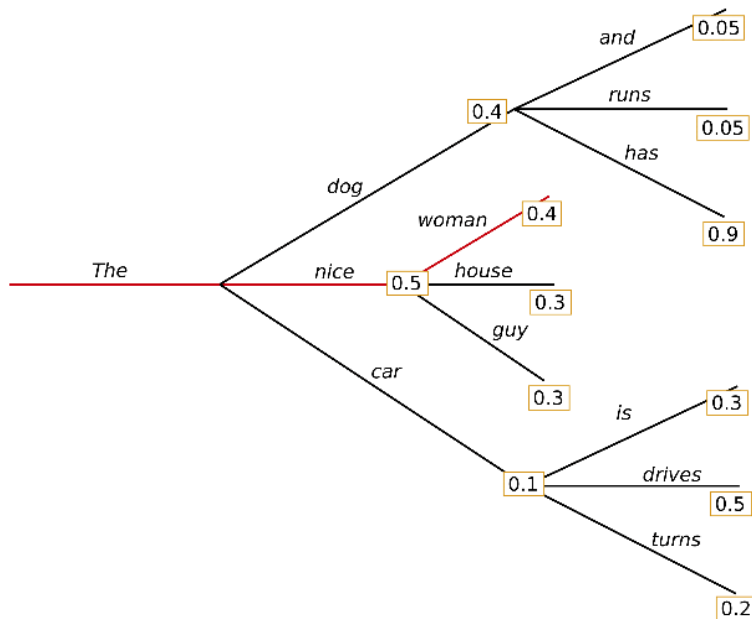
2. Beam Search



Decoding Strategies (1) – Greedy Search

1. Greedy Search

- 각 시점에서 **가장 높은 확률을 가진 단어를 선택**하여 다음 단어를 선택하는 것
- 전체적인 문장의 일관성이나 자연스러움을 고려하지 않음
- 문장 생성시 반복이 빠르게 생김

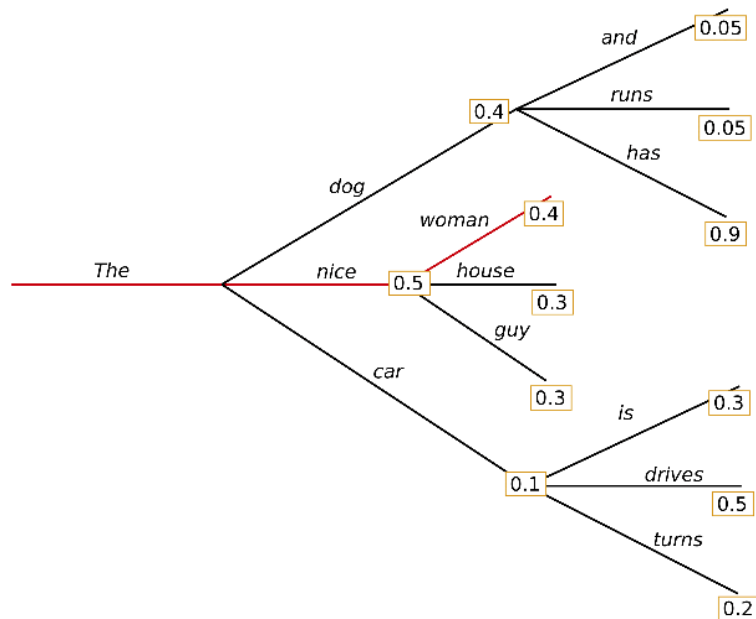


Greedy 알고리즘의 대표적 문제 상황

Decoding Strategies (1) – Beam Search

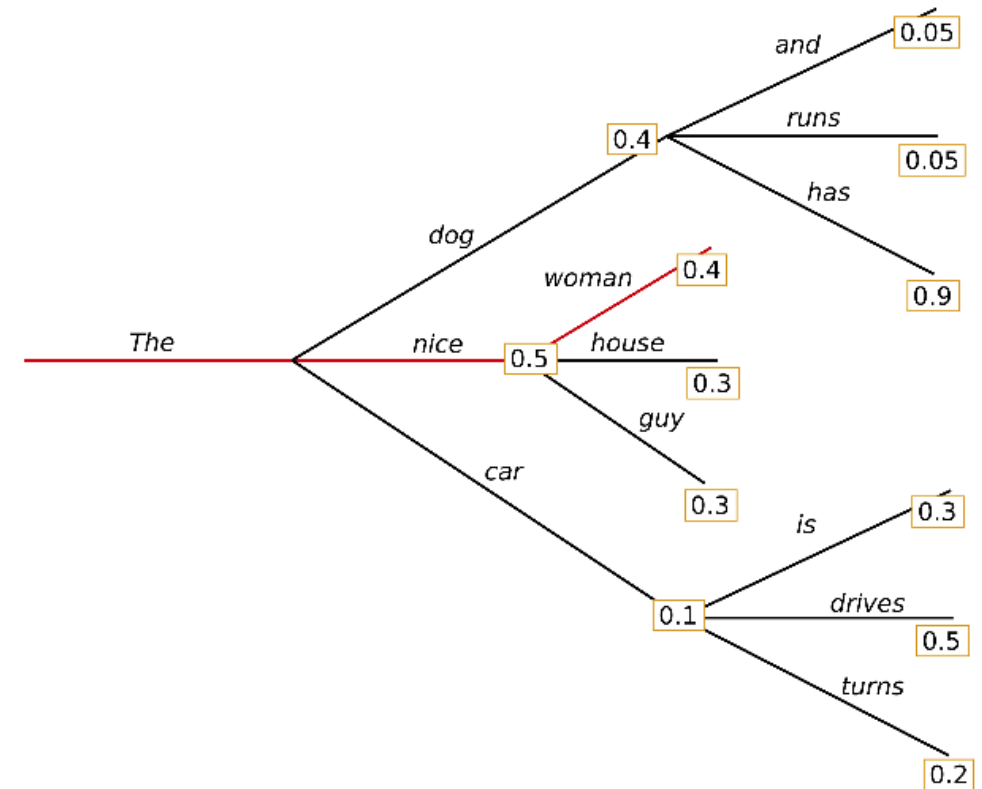
2. Beam Search

- 각 시점에서 **여러 후보 단어를 유지**하며, 각 후보 단어의 확률을 고려하여 다음 단어를 선택
- 매 시점마다 최선의 후보를 추적하여 전체 문장의 일관성을 높이려고 함
- 더 많은 후보를 유지한다면, 다양성이 늘어나지만 계산 비용이 더 많이 들 수 있음



Decoding Strategies (1) – Beam Search

- Example
 - 유지할 후보의 수: 2
 - Step 1) The
 - Step 2) nice (0.5), dog (0.4)
 - Step 3)
 - nice (0.5) $\rightarrow$ woman (0.4), house (0.3), guy (0.3)
 - dog (0.4) $\rightarrow$ has (0.9), runs (0.05), and (0.05)
 - $\rightarrow$ The dog has (0.36), The nice woman (0.20)
 - ...



Decoding Strategies (2) – 확률적 / Sampling Based

Prompts

 Your prompts 

Save

Model

gpt-4.1 

text.format: text temp: 1.00 tokens: 2048 top_p: 1.00 store: true



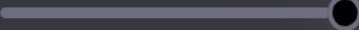
[OpenAI Playground](#)

Text format

text 

Temperature

1.00



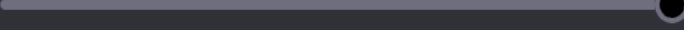
Max tokens

2048



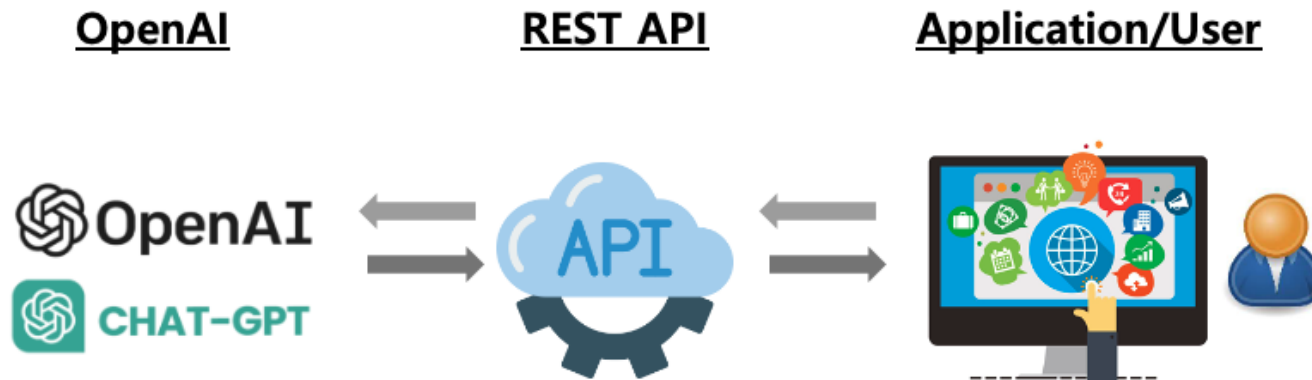
Top P

1.00



Store logs





Decoding Strategies (2) – 확률적 / Sampling Based

- Search 기반 방법과는 다르게, **확률적으로 token을 선택하는 방식**
(높은 확률이라고 해서 무조건 뽑는 것이 아니라, 확률에 따라 sampling을 하는 것!)
- 대표적으로 두 가지 전략이 존재함
 1. 확률 분포 후보군을 필터링하자 → **Top-k / Top-p**
 2. 확률 분포의 모양을 조절하자 → **Temperature**

Decoding Strategies (2) – Top-k / Top-p

1. Top-k / Top-p

- Sampling 후보군을 필터링을 통해 제한하는 방법
- Top-k
 - 확률이 높은 상위 **k개 토큰만** 남기기
- Top-p
 - 누적 확률이 **p 이상**이 될 때까지 **token을 선택**하기

➔ 확률분포 기반 방법의 단점인 무작위성을 통제하는 방법

The dress color was _____
 $P(* | \text{The dress color was})$

red	0.03	■
white	0.03	■
black	0.02	■
pink	0.02	■
blue	0.02	■
...	...	
violet	0.02	■
...	...	
olive	0.02	■
...	...	

Top-4

Top-k sampling

The dress color was _____
 $P(* | \text{The dress color was})$

red	0.03	■
white	0.03	■
black	0.02	■
pink	0.02	■
blue	0.02	■
...	...	
violet	0.02	■
...	...	
olive	0.02	■
...	...	

Top-80%

Top-p sampling

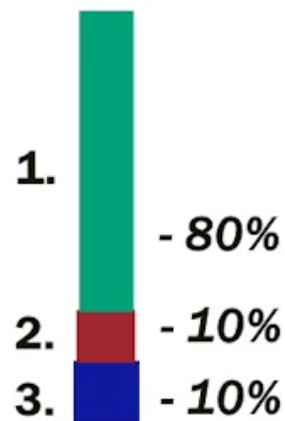
Decoding Strategies (2) – 확률적 / Sampling Based

2. Temperature

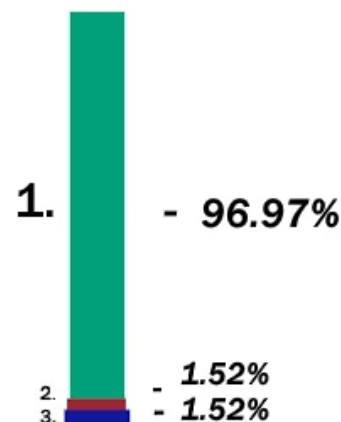
- 확률 분포의 모양을 조절하여 다양성을 제어하는 방법
- 샘플링 전 확률 분포 자체를 날카롭게($T < 1.0$) 또는 평평하게($T > 1.0$) 조절
- Temperature T
 - $T = 1.0$
 - $T < 1.0$: 다양성 감소
 - $T > 1.0$: 다양성 증가

(original probabilities)

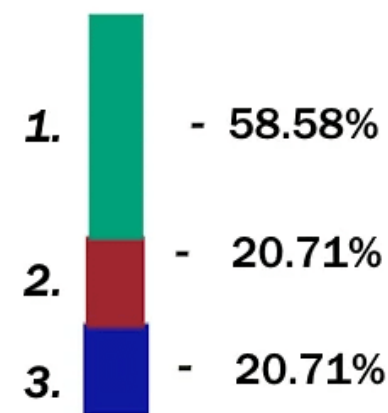
1.0 temp



0.5 temp



2.0 temp



Decoding Strategies (2) – Temperature

Use Case	Temperature	Top_p	Description
Code Generation	0.2	0.1	Generates code that adheres to established patterns and conventions. Output is more deterministic and focused. Useful for generating syntactically correct code.
Creative Writing	0.7	0.8	Generates creative and diverse text for storytelling. Output is more exploratory and less constrained by patterns.
Chatbot Responses	0.5	0.5	Generates conversational responses that balance coherence and diversity. Output is more natural and engaging.
Code Comment Generation	0.3	0.2	Generates code comments that are more likely to be concise and relevant. Output is more deterministic and adheres to conventions.
Data Analysis Scripting	0.2	0.1	Generates data analysis scripts that are more likely to be correct and efficient. Output is more deterministic and focused.
Exploratory Code Writing	0.6	0.7	Generates code that explores alternative solutions and creative approaches. Output is less constrained by established patterns.

출처

Example → 각자의 Use Case에 맞는 다양한 조합이 가능

Decoding Strategies (3) – 반복 제어 전략

- 자연어를 생성할 때, 가장 높은 확률의 토큰만 선택한다면 **동일한 단어나 구문이 반복** 생성될 수 있음.
- 따라서, 이런 **반복을 방지하기 위한 전략**들이 존재:
 1. N-gram penalty
 2. Token-based penalty

Decoding Strategies (3) – N-gram penalty

- **N-gram**
 - 자연어 처리에서 **연속된 N개의 단어(token) 묶음**을 의미함
 - ex) "I love natural language processing"을 2-grams으로 나누면 "I love", "love natural", "natural language", "language processing"과 같은 단어 쌍들이 생성
- **N-gram penalty**: 특정 n-gram 패턴이 반복되어 나타나는 것을 제한하는 것
 - 다양성을 유지할 수 있고, 지루함을 줄임
 - 반복되는 패턴을 줄이면서 자연스러움

Decoding Strategies (3) – Token-based penalty

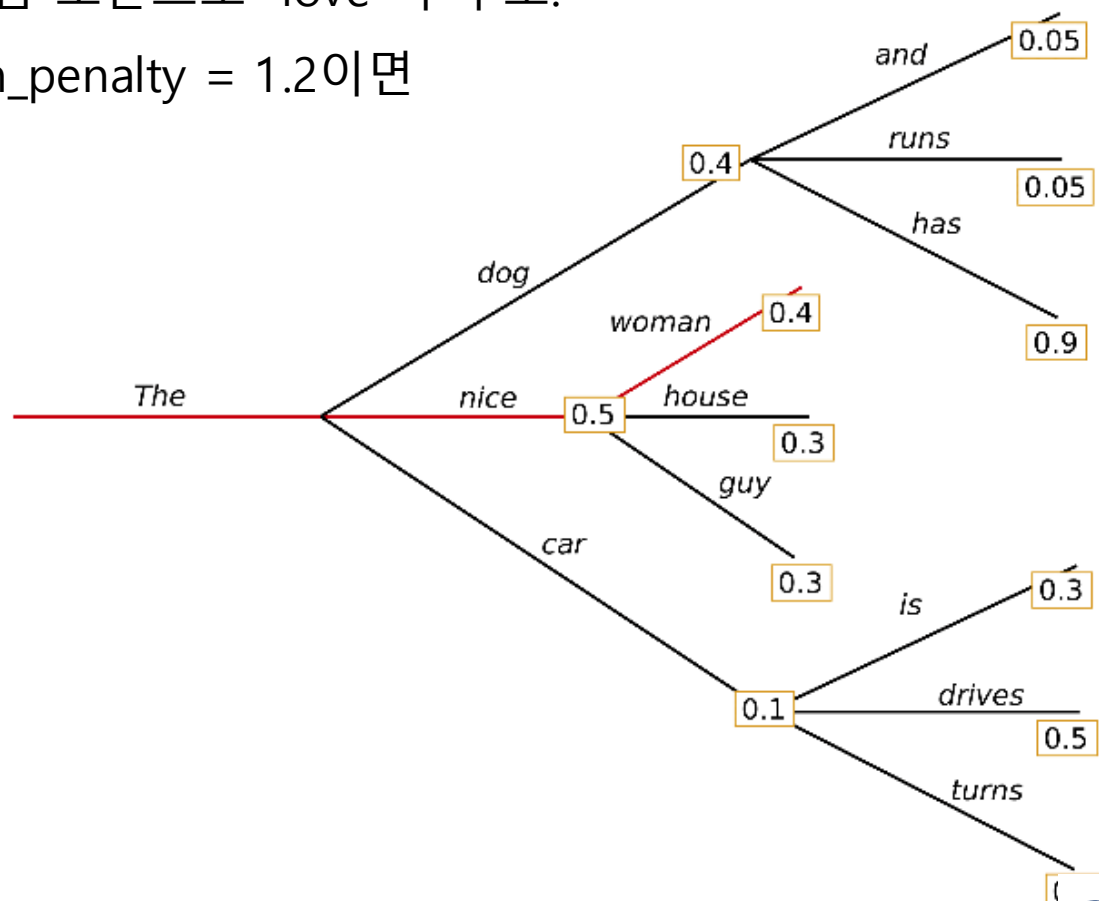
- 이미 생성된 토큰이 다시 등장하려 할 때, 해당 토큰의 확률(logit 값)에 **penalty**를 적용하여 생성 가능성을 낮추는 전략

- ex) 문장 중 "love"가 이미 등장한 상황이고, 다음 토큰으로 "love"가 후보.

원래의 logit 값: $\text{logit}(\text{"love"}) = 3.0$, $\text{repetition_penalty} = 1.2$ 이면

조정된 $\text{logit}(\text{"love"}) = 3.0 / 1.2 \approx 2.5$

➔ "love"의 생성 확률 낮아짐



GPT 실습 – Decoding Strategy 점검

- GPT-2를 이용해, 다양한 조합의 Decoding Strategy를 적용해보고, setting 별로 어떤 생성 결과가 나오는지 확인해보기

- ✓ Decoding Strategy 점검

- a) 결정적 / Search Based

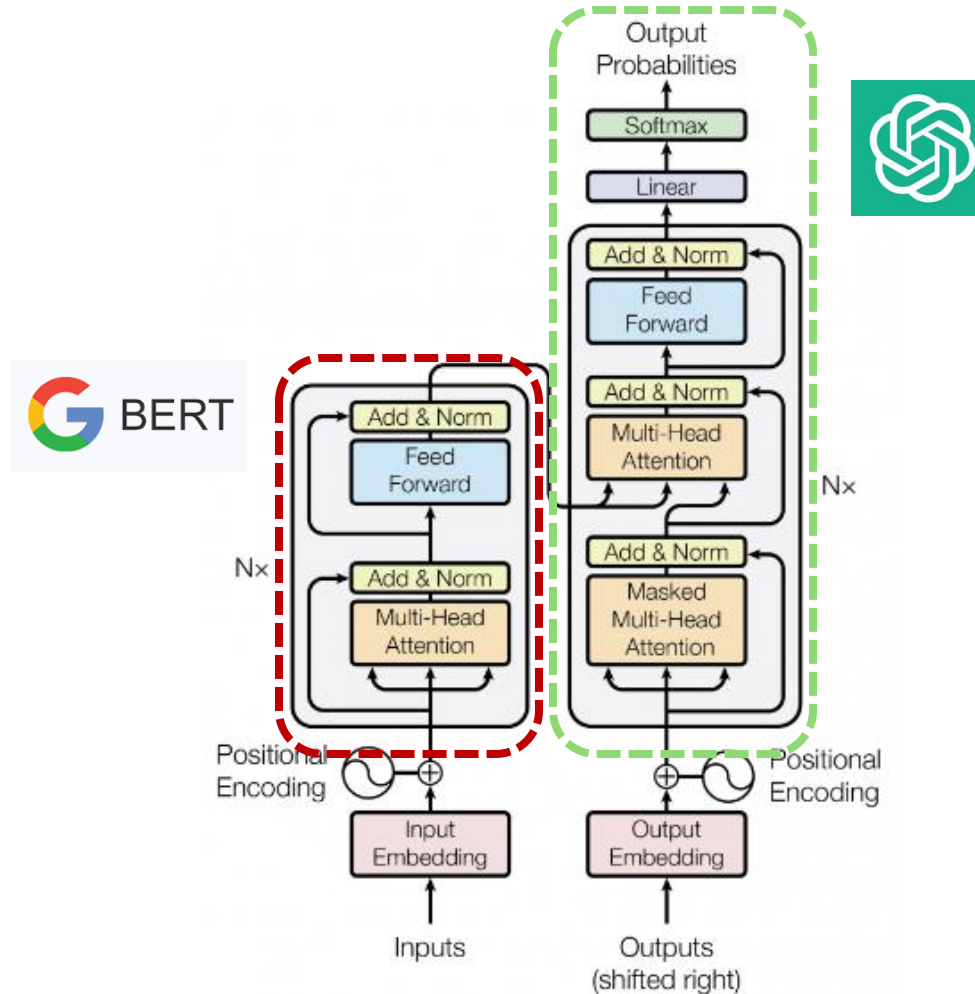
- [] ↳ 숨겨진 셀 11개

- b) 확률적 / Sampling Based

- [] ↳ 숨겨진 셀 1개

BERT vs GPT – 성능 비교해보기

- Sentiment Analysis Task와 Question Answering Task에서 각각 **BERT**와 **GPT**의 성능 차이를 확인 해보기



최근의 Language Models

최근의 Language Models

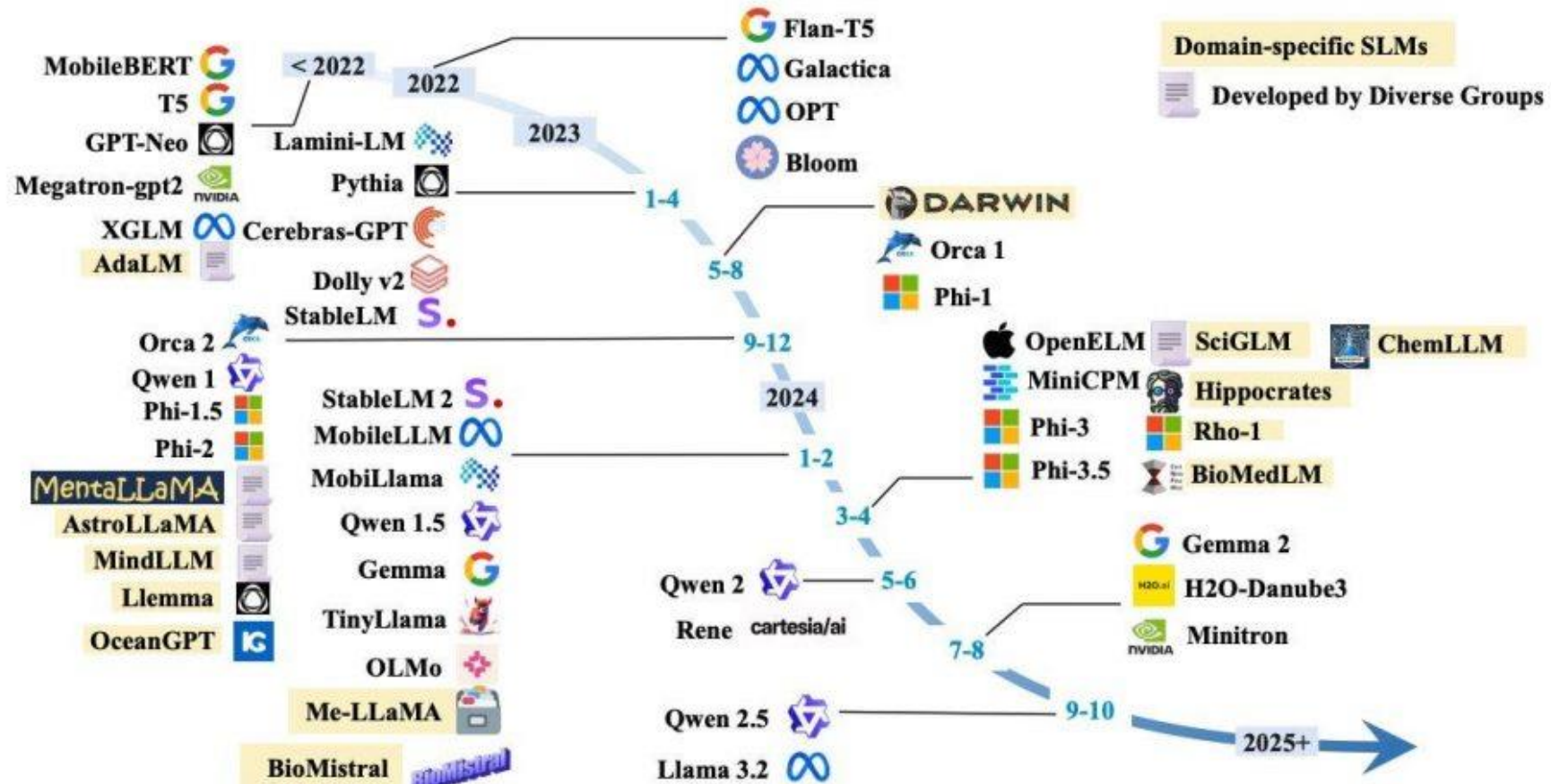
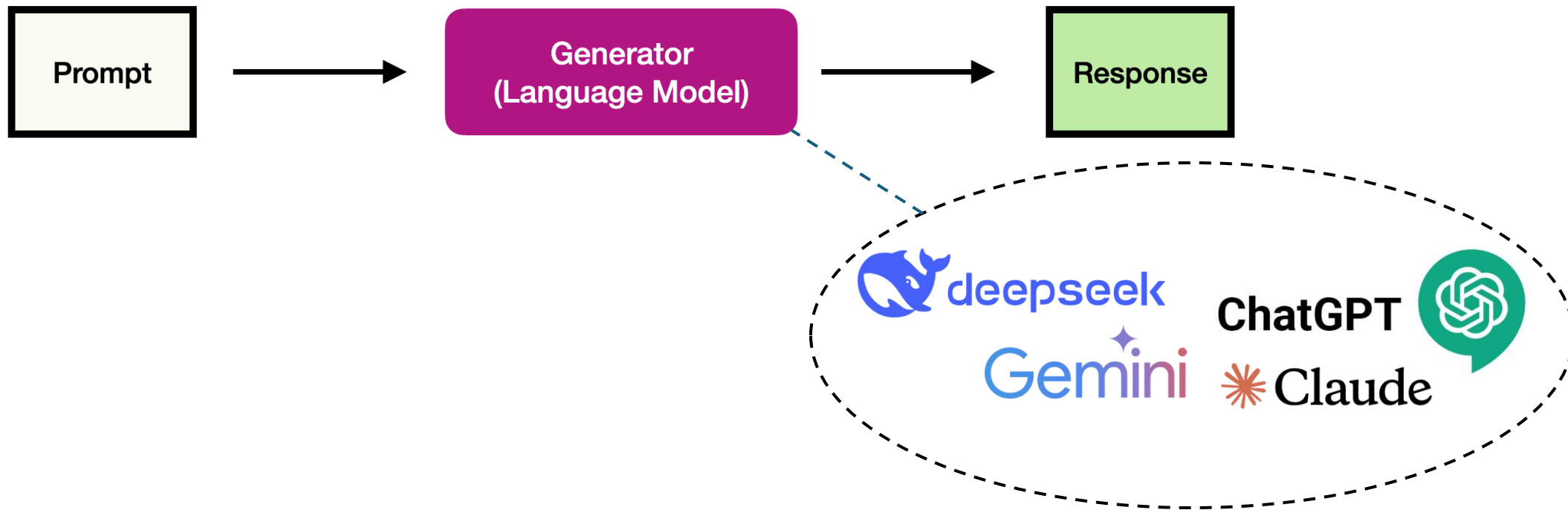
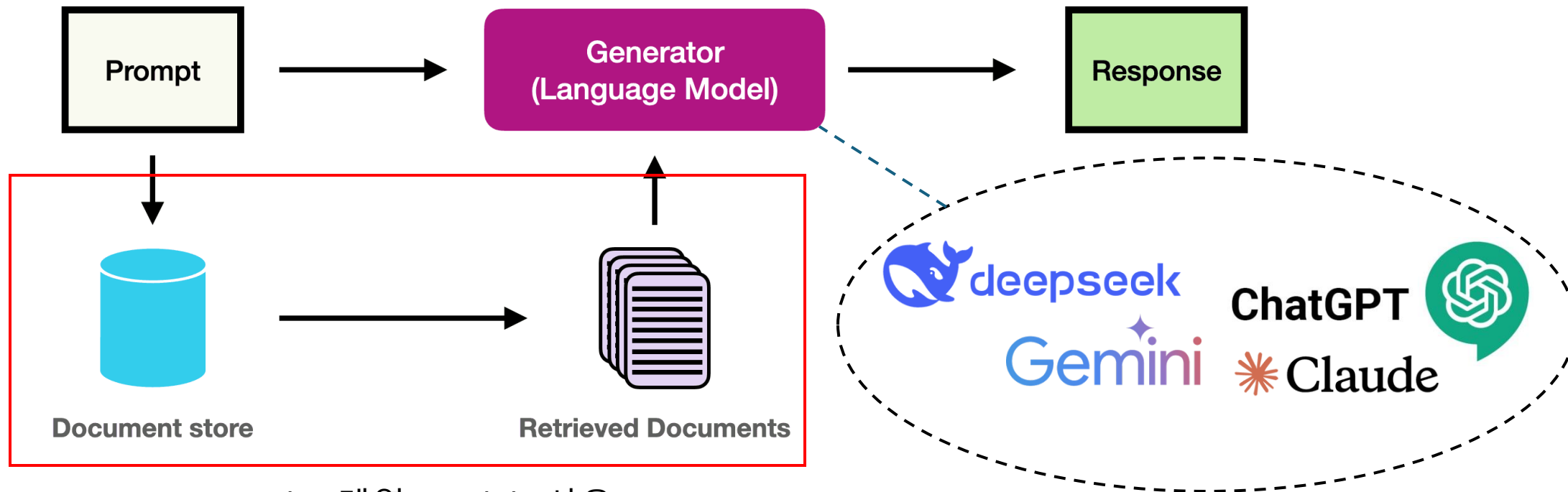


Fig. 3. A timeline of existing small language models.



Retrieval Augmented Generation



Encoder 계열 Models 사용

Next Lecture: Large Language Models (LLMs) & Prompting

Thank you!